
● SAVOL 41:

NUMPY DA MATRITSALAR BILAN ISHLASH: DOT PRODUCT , TRANSPOSE , INVERSE . CHIZIQLI ALGEBRA AMALLARINI ML DA QO'LLASH VA REGRESSIYA KOEFFITSIYENTLARINI ANALITIK HISOBLASH.

1. Nima Uchun Chiziqli Algebra ML Da Muhim?

ML algoritmlarining matematikasi:

Chiziqli regressiya: $\beta = (X^T X)^{-1} X^T y$

Neyron tarmoq: $Z = WX + b$

PCA: $\Sigma = (1/n) X^T X$ (kovaryans matritsasi)

SVM: $\max W$ bilan dot product

K-Means: $\|x - \mu\|^2 = \text{masofa}$

Barchasi matritsa amallari!

NumPy → Bu amallarni tez va oson bajaradi

2. NumPy Array — Asosiy Tushuncha

```
import numpy as np
```

```
# Vektor (1D):
```

```
v = np.array([1, 2, 3])  
print(v.shape)    # (3,)
```

```
# Matritsa (2D):
```

```
A = np.array([[1, 2, 3],  
              [4, 5, 6]])  
print(A.shape)    # (2, 3) → 2 qator, 3 ustun
```

```
# 3D tensor:
```

```
T = np.zeros((batch, qatlam, o'lcham))
```

```
# Muhim atributlar:
```

```
print(A.ndim)     # 2 → O'lchamlar soni  
print(A.size)     # 6 → Jami elementlar  
print(A.dtype)    # float64 → Ma'lumot turi
```

Array yaratish usullari:

```
# Nollar matritsasi:
```

```
np.zeros((3, 4))    # 3×4, barchasi 0
```

```
# Birlar matritsasi:
```

```
np.ones((2, 3))     # 2×3, barchasi 1
```

```
# Birlik matritsasi:
```

```
np.eye(3)           # 3×3, diagonal=1
```

```
# [[1, 0, 0],
# [0, 1, 0],
# [0, 0, 1]]

# Tasodifiy:
np.random.randn(3, 3)    # Normal taqsimot
np.random.rand(3, 3)     # Uniform [0,1]

# Diapazon:
np.arange(0, 10, 2)      # [0, 2, 4, 6, 8]
np.linspace(0, 1, 5)     # [0, 0.25, 0.5, 0.75, 1.0]
```

3. Transpose — Ko'chirish

Nazariy asos:

$$(A)^T_{ij} = A_{ji}$$

Qatorlar ↔ Ustunlar almashadi

```
A = [[1, 2, 3],
      [4, 5, 6]]
A.T = [[1, 4],
        [2, 5],
        [3, 6]]
A.shape = (2, 3)    A.T.shape = (3, 2)
```

```
A = np.array([[1, 2, 3],
              [4, 5, 6]])
```

Transpose usullari:

```
A_t1 = A.T           # Atribut
A_t2 = A.transpose() # Metod
A_t3 = np.transpose(A) # Funksiya
```

```
print(A_t1)
```

```
# [[1, 4],
# [2, 5],
# [3, 6]]
```

Vektorlar uchun muhim nuans:

```
v = np.array([1, 2, 3])
print(v.shape)    # (3,) → 1D!
print(v.T.shape)  # (3,) → 0'zgarmadi!
```

To'g'ri usul — 2D qilish:

```
v_col = v.reshape(-1, 1)    # (3, 1) → Ustun vektor
v_row = v.reshape(1, -1)    # (1, 3) → Qator vektor
print(v_col.T.shape)        # (1, 3) ✓
```

ML da Transpose:

Kovaryans matritsasi:

```
X = np.random.randn(100, 5)    # 100 namuna, 5 xususiyat
cov = (1/100) * X.T @ X        # (5,5) kovaryans
# X.T: (5,100), X: (100,5) → X.T @ X: (5,5) ✓
```

Gradient descent:

```
# ∂L/∂W = X.T · (ŷ - y) / n
grad_W = X.T @ (y_pred - y) / len(y)
```

4. Dot Product — Skalyar Ko'paytma

Nazariy asos:

Vektor dot product:

$$a \cdot b = \sum a_i b_i = a_1 b_1 + a_2 b_2 + \dots + a_n b_n$$

$$a \cdot b = ||a|| \cdot ||b|| \cdot \cos(\theta)$$

Geometrik ma'no:

$$a \cdot b = 0 \rightarrow \text{Vektorlar perpendikulyar (ortogonal)}$$

$$a \cdot b > 0 \rightarrow \text{Bir tomonga yo'nalgan } (\theta < 90^\circ)$$

$$a \cdot b < 0 \rightarrow \text{Qarama-qarshi tomonda } (\theta > 90^\circ)$$

Matritsa ko'paytmasi:

$$(AB)_{ij} = \sum_k A_{ik} B_{kj}$$

$$A: (m \times k), B: (k \times n) \rightarrow AB: (m \times n)$$

Ichki o'lchamlar teng bo'lishi shart!

Vektor dot product:

```
a = np.array([1, 2, 3])
```

```
b = np.array([4, 5, 6])
```

```
d1 = np.dot(a, b)      # 1×4 + 2×5 + 3×6 = 32
```

```
d2 = a @ b            # Python 3.5+ operator
```

```
d3 = (a * b).sum()     # Element-wise × keyin yig'ish
```

```
# Barchasi: 32
```

Matritsa ko'paytmasi:

```
A = np.array([[1, 2],  
              [3, 4]]) # (2,2)
```

```
B = np.array([[5, 6],  
              [7, 8]]) # (2,2)
```

```
C = A @ B # yoki np.dot(A, B)
```

```
# [[1×5+2×7, 1×6+2×8], [[19, 22],
```

```
# [3×5+4×7, 3×6+4×8]] = [43, 50]]
```

O'lcham tekshiruvi — Muhim:

```
# (m×k) @ (k×n) = (m×n)
```

```
A = np.ones((3, 4)) # (3,4)
```

```
B = np.ones((4, 5)) # (4,5)
```

```
C = A @ B           # (3,5) ✓
```

Noto'g'ri:

```
D = np.ones((3, 4))
```

```
E = np.ones((3, 5))
```

```
# D @ E → ValueError! 4 ≠ 3
```

ML da:

```
X = np.ones((100, 10)) # 100 namuna, 10 xususiyat
```

```
W = np.ones((10, 1))   # 10 ta vazn, 1 ta chiqish
```

```
y = X @ W              # (100, 1) bashoratlar ✓
```

ML da Dot Product:

```

# Neyron tarmoq bir qatlami:
def forward(X, W, b):
    return X @ W + b
    # X: (batch, kirish)
    # W: (kirish, chiqish)
    # b: (chiqish,)
    # Natija: (batch, chiqish)

# Kosinusli o'xshashlik:
def cosine_similarity(a, b):
    return np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))
    # NLP da so'z vektorlari taqqoslash

```

5. Inverse — Teskari Matritsa

Nazariy asos:

$$A \times A^{-1} = A^{-1} \times A = I \text{ (Birlik matritsa)}$$

Mavjudlik sharti:

- Faqat KVADRAT matritsa ($n \times n$)
- Determinant $\neq 0$ ($\det(A) \neq 0$)
- "Singular bo'lmagan" matritsa

$\det(A) = 0 \rightarrow$ Teskari mavjud emas (Singular matritsa)

$\rightarrow$ Qatorlar chiziqli bog'liq (multikolinearlik!)

2x2 holat:

$$A = \begin{bmatrix} a & b \\ c & d \end{bmatrix}, \quad A^{-1} = \frac{1}{\det(A)} \times \begin{bmatrix} d & -b \\ -c & a \end{bmatrix}$$

$$\det(A) = ad - bc$$

```
import numpy as np
```

```
A = np.array([[4, 7],
               [2, 6]])
```

Determinant:

```
det = np.linalg.det(A)
```

```
print(f"det(A) = {det:.1f}")    # 10.0 ≠ 0 → Teskari mavjud
```

Teskari matritsa:

```
A_inv = np.linalg.inv(A)
```

```
print(A_inv)
```

```
# [[ 0.6, -0.7],
```

```
# [-0.2,  0.4]]
```

Tekshirish: $A \times A^{-1} = I$

```
I = A @ A_inv
```

```
print(np.round(I, 10))
```

```
# [[1., 0.],
```

```
# [0., 1.]] ← Birlik matritsa ✓
```

Singular matritsa (det=0):

```
B = np.array([[1, 2],
               [2, 4]])    # 2-qator = 2 × 1-qator
```

```
det_B = np.linalg.det(B)    # ≈ 0
# np.linalg.inv(B) → LinAlgError: Singular matrix!
```

Pseudoinverse — Singular holatlar uchun:

```
# Moore-Penrose Pseudoinverse:
# Har qanday matritsa uchun ishlaydi (kvadrat bo'lmasa ham!)
A_pinv = np.linalg.pinv(A)

# Nima uchun kerak:
# Chiziqli regressiyada  $n < p$  bo'lsa (namuna < xususiyat)
# →  $X^T X \rightarrow \text{Singular} \rightarrow \text{inv}()$  ishlamaydi
# →  $\text{pinv}()$  → Yechim topadi (eng kichik normali)
```

6. OLS — Regressiya Koeffitsiyentlarini Analitik Hisoblash

Chiziqli regressiya modeli:

$$y = X\beta + \varepsilon$$

X : ($n \times p$) ma'lumot matritsasi
 y : ($n \times 1$) maqsad vektor
 β : ($p \times 1$) koeffitsiyentlar (topish kerak!)
 ε : ($n \times 1$) xatolar

OLS maqsadi:

$$\min ||y - X\beta||^2 = \min \sum (y_i - X_i\beta)^2$$

Analitik yechim (Normal tenglamalar):

$$\begin{aligned} \partial/\partial\beta [||y - X\beta||^2] &= 0 \\ -2X^T(y - X\beta) &= 0 \\ X^T y &= X^T X \beta \\ (X^T X) \beta &= X^T y \\ \beta &= (X^T X)^{-1} X^T y \leftarrow \text{Normal tenglamalar yechimi!} \end{aligned}$$

Qadamba-qadam hisoblash:

```
import numpy as np
```

```
def ols_manual(X, y):
```

```
    """
    OLS koeffitsiyentlarini analitik hisoblash
     $\beta = (X^T X)^{-1} X^T y$ 
    """
```

```
    # Bias uchun birlar ustuni qo'shish:
```

```
    n = X.shape[0]
    X_b = np.column_stack([np.ones(n), X])
    # X_b: (n, p+1) →  $\theta_0$  uchun +1 ustun
```

```
    # Bosqichma-bosqich:
```

```
    Xt = X_b.T                                # Transpose: (p+1, n)
    XtX = Xt @ X_b                             # (p+1, p+1)
    XtX_inv = np.linalg.inv(XtX)               # (p+1, p+1)
    Xty = Xt @ y                               # (p+1,)
    beta = XtX_inv @ Xty                       # (p+1,) ← Natija!
```

```
    return beta
```

```

# Misol – Uy narxi:
np.random.seed(42)
n = 100

maydon    = np.random.uniform(50, 200, n)    # m²
xonalar   = np.random.randint(2, 6, n)       # ta
masofa    = np.random.uniform(1, 20, n)      # km

# Haqiqiy model: narx = 1000 + 2*maydon + 50*xonalar - 30*masofa + shovqin
narx = (1000 + 2*maydon + 50*xonalar
        - 30*masofa + np.random.randn(n)*50)

X = np.column_stack([maydon, xonalar, masofa])
y = narx

# OLS hisoblash:
beta = ols_manual(X, y)

print("Topilgan koeffitsiyentlar:")
print(f"  β₀ (intercept): {beta[0]:.2f} (haqiqiy: 1000)")
print(f"  β₁ (maydon):      {beta[1]:.2f} (haqiqiy: 2)")
print(f"  β₂ (xonalar):      {beta[2]:.2f} (haqiqiy: 50)")
print(f"  β₃ (masofa):      {beta[3]:.2f} (haqiqiy: -30)")

# Sklearn bilan taqqoslash:
from sklearn.linear_model import LinearRegression
lr = LinearRegression()
lr.fit(X, y)
print(f"\nSklearn: {[lr.intercept_] + list(lr.coef_)}")
# Deyarli bir xil! ✓

```

7. OLS Muammolari va Yechimlar

Muammo 1 — $X^T X$ Singular:

```

# Multikolinearlik →  $\det(X^T X) \approx 0 \rightarrow \text{inv}()$  ishlamaydi

# Yechim 1 – Pseudoinverse:
def ols_pinv(X, y):
    X_b = np.column_stack([np.ones(len(X)), X])
    beta = np.linalg.pinv(X_b) @ y
    return beta

# Yechim 2 – SVD (Singular Value Decomposition):
def ols_svd(X, y):
    X_b = np.column_stack([np.ones(len(X)), X])
    #  $X_b = U \Sigma V^T \rightarrow \theta = V \Sigma^{-1} U^T y$ 
    U, S, Vt = np.linalg.svd(X_b, full_matrices=False)
    beta = Vt.T @ np.diag(1/S) @ U.T @ y
    return beta

# Yechim 3 – Ridge (L2 regularizatsiya):
def ridge_analitik(X, y, alpha=1.0):
    X_b = np.column_stack([np.ones(len(X)), X])

```

```

p = X_b.shape[1]
#  $(X^T X + \alpha I)^{-1} X^T y$ 
beta = np.linalg.inv(X_b.T @ X_b + alpha*np.eye(p)) @ X_b.T @ y
return beta
#  $\alpha I \rightarrow X^T X$  ga qo'shiladi  $\rightarrow$  Hech qachon singular bo'lmaydi!

```

Muammo 2 — Katta n va p:

$n=1,000,000$ namuna, $p=1,000$ xususiyat:
 $X^T X \rightarrow (1000 \times 1000)$ matritsa
 $\text{inv}(X^T X) \rightarrow O(p^3) = O(10^9)$ operatsiya $\rightarrow$ Juda sekin!

Yechim $\rightarrow$ Gradient Descent:
 Analitik: $\beta = (X^T X)^{-1} X^T y \rightarrow$ Katta n da sekin
 GD: Iterativ yangilash $\rightarrow$ Katta n da tezroq

Qoida:
 n va p kichik ($<10,000$) $\rightarrow$ Analitik (OLS)
 n yoki p katta $\rightarrow$ Gradient Descent

8. Muhim NumPy Chiziqli Algebra Funksiyalari

```

# Determinant:
np.linalg.det(A)

# Inverse:
np.linalg.inv(A)

# Pseudoinverse:
np.linalg.pinv(A)

# Eigenqiymatlar:
eigenvalues, eigenvectors = np.linalg.eig(A)

# SVD:
U, S, Vt = np.linalg.svd(A)

# Norm:
np.linalg.norm(v)           # L2 (Evklid)
np.linalg.norm(v, ord=1)    # L1 (Manhattan)
np.linalg.norm(A, 'fro')    # Frobenius (matritsa)

# Chiziqli tenglamalar sistemasi:  $Ax = b$ 
x = np.linalg.solve(A, b)   #  $\text{inv}(A) @ b$  dan tezroq!

# Rang:
np.linalg.matrix_rank(A)

# Shart soni (konditsion raqam):
np.linalg.cond(A)
# Katta  $\rightarrow$  Matritsa "deyarli singular"  $\rightarrow$  Beqaror!

```

Konditsion raqam — Muhim tushuncha:

$$\kappa(A) = ||A|| \cdot ||A^{-1}||$$

$\kappa = 1 \rightarrow$ Ideal (birlik matritsa)
 $\kappa = 10 \rightarrow$ Yaxshi
 $\kappa = 10^6 \rightarrow$ Yomon (beqaror)
 $\kappa = 10^{10} \rightarrow$ Juda yomon $\rightarrow$ Natijalar ishonchsiz

ML da:

$\kappa(X^T X)$ katta $\rightarrow$ Multikolinearlik muammosi
Ridge: α qo'shish $\rightarrow \kappa$ kamayadi $\rightarrow$ Barqaror

9. Broadcasting — Kuchli Mexanizm

Broadcasting: Turli o'lchamdagi arraylar bilan amallar

Har qatordan o'rtachani ayirish (normalizatsiya):

```
X = np.random.randn(100, 5)    # (100, 5)
mean = X.mean(axis=0)           # (5,)  $\rightarrow$  Har ustun o'rtachasi
X_centered = X - mean           # (100,5) - (5,)  $\rightarrow$  Broadcasting!
# NumPy (5,)  $\rightarrow$  (1, 5)  $\rightarrow$  (100, 5) ga kengaytiradi
```

Standartlashtirish:

```
std = X.std(axis=0)              # (5,)
X_scaled = (X - mean) / std      #  $\leftarrow$  StandardScaler qo'lda!
```

Masofa matritsasi:

$\|x_i - x_j\|^2$ ni barcha juftlar uchun

```
def pairwise_distances(X):
    # X: (n, p)
    # Natija: (n, n)
    diff = X[:, np.newaxis, :] - X[np.newaxis, :, :]
    # (n,1,p) - (1,n,p)  $\rightarrow$  (n,n,p)
    return np.sqrt((diff**2).sum(axis=-1))
# K-Means, KNN da ishlatiladi
```

10. Xulosa

Transpose (A^T):

Qatorlar $\leftrightarrow$ Ustunlar

ML: Kovaryans, gradient hisoblash

$v.T \rightarrow 1D$ da o'zgarmaydi $\rightarrow$ reshape(-1,1) kerak

Dot Product ($A @ B$):

$(m \times k) @ (k \times n) = (m \times n)$

ML: Forward pass, bashorat, o'xshashlik

Inverse (A^{-1}):

$AA^{-1} = I$

Faqat kvadrat, $\det \neq 0$

Singular $\rightarrow$ pinv() yoki SVD

OLS Analitik:

$\beta = (X^T X)^{-1} X^T y$

Kichik dataset $\rightarrow$ Tez va aniq

Katta dataset $\rightarrow$ Gradient Descent afzal

Muhim funksiyalar:

np.linalg.inv() → Teskari
np.linalg.solve() → $Ax=b$ (inv dan tezroq)
np.linalg.svd() → Decomposition
np.linalg.pinv() → Pseudoinverse
np.linalg.cond() → Barqarorlik tekshiruvi

Qoidalar:

- ✓ @ operatori → Matritsa ko'paytmasi
- ✓ solve() → inv() @ b dan aniqroq
- ✓ Konditsion raqam tekshir → Multikolinearlik
- ✓ Ridge → Singular muammosini hal qiladi

💡 **Eslab qol:** $\beta = (X^T X)^{-1} X^T y$ — bu formula chiziqli regressiyaning **butun matematikasini** bitta qatorda ifodalaydi. Sklearn ning LinearRegression.fit() ichida aynan shu formula ishlaydi! NumPy bilan bu formulani yozish — ML matematikasini **haqiqiy tushunganlik** belgisi!

● SAVOL 42:

PANDAS DA VAQT QATORI (TIME SERIES) MA'LUMOTLARI BILAN ISHLASH: DATETIME INDEKS, RESAMPLE , ROLLING WINDOW . SAVDO YOKI OB-HAVO MA'LUMOTLARINI TAHLIL QILISHDA QO'LLASH.

1. Vaqt Qatori — Nazariy Asos

Vaqt qatori (Time Series):

Vaqt bo'yicha tartiblab qayd etilgan kuzatuvlar ketma-ketligi.
Har kuzatuv → Aniq vaqt belgisi (timestamp)

Oddiy ma'lumotdan farqi:

Oddiy: Kuzatuvlar mustaqil (tartib muhim emas)
Vaqt qatori: Kuzatuvlar bog'liq (tartib MUHIM!)
Kecha bugungiga ta'sir qiladi

Vaqt qatori komponentlari:

Trend: Uzoq muddatli o'sish/kamayish yo'nalishi
Mavsumiylik: Davriy (haftalik, oylik, yillik) naqsh
Tsikl: Uzoq muddatli, davriy bo'lmagan tebranish
Shovqin: Tasodifiy o'zgarishlar

Hayotiy misollar:

- Aksiya narxlari (kunlik)
 - Temperatura (soatlik)
 - Elektr sarfi (daqiqalik)
 - Veb sayt tashrifi (sekundlik)
 - GDP (yillik)
-

2. Datetime — Vaqt Belgilari

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Datetime yaratish:
t1 = pd.Timestamp("2024-01-15")
t2 = pd.Timestamp("2024-01-15 14:30:00")
t3 = pd.to_datetime("15/01/2024", dayfirst=True)

# DatetimeIndex — Vaqt indeksi:
indeks = pd.date_range(
    start="2024-01-01",
    end="2024-12-31",
    freq="D"          # D=kunlik, H=soatlik, M=oylik, W=haftalik
)
print(f"Kunlar soni: {len(indeks)}")  # 366

# Freq parametrlari:
# "D" → Kunlik
# "H" → Soatlik
# "T" → Daqiqalik (yoki "min")
# "S" → Sekundlik
# "W" → Haftalik (Yakshanba)
# "M" → Oyni oxiri
# "MS" → Oyni boshi
# "Q" → Kvartal oxiri
# "A" → Yil oxiri
# "B" → Ish kunlari (Monday-Friday)
# "2D" → Har 2 kun

# String → Datetime:
df = pd.DataFrame({
    "sana": ["2024-01-01", "2024-01-02", "2024-01-03"],
    "qiymat": [100, 105, 98]
})
df["sana"] = pd.to_datetime(df["sana"])
df = df.set_index("sana")

# Datetime xususiyatlari:
df.index.year      # Yil
df.index.month     # Oy
df.index.day       # Kun
df.index.hour      # Soat
df.index.weekday   # Hafta kuni (0=Dushanba)
df.index.quarter   # Kvartal (1-4)
```

3. Vaqt Bo'yicha Filtrlash

```
# Real savdo ma'lumoti yaratish:
np.random.seed(42)
kunlar = pd.date_range("2022-01-01", "2024-12-31", freq="B")
# "B" → Faqat ish kunlari
```

```

narx_qiymatlar = 1000 + np.cumsum(np.random.randn(len(kunlar)) * 20)
# Cumsum → Tasodifiy yurish (Random Walk) → Aksiya narxiga o'xshash

aksiya = pd.DataFrame({
    "narx": narx_qiymatlar,
    "hajm": np.random.randint(100000, 1000000, len(kunlar))
}, index=kunlar)

# Vaqt bo'yicha filtrlash – Pandas ning kuchli tomoni:

# Yil bo'yicha:
aksiya_2023 = aksiya["2023"] # 2023 yil
aksiya_q1 = aksiya["2023-01":"2023-03"] # 1-kvartal

# Aniq sana:
bir_kun = aksiya["2023-06-15"]

# Oraliq:
oraliq = aksiya["2023-01-01":"2023-06-30"]

# .loc bilan:
aksiya.loc["2023-03-15":"2023-03-31"]

# Hafta kuni filtrlash:
dushanba = aksiya[aksiya.index.weekday == 0]
ish_kunlari = aksiya[aksiya.index.weekday < 5]

```

4. Resample — Davrni O'zgartirish

Resample – Vaqt qatorini boshqa chastotaga o'zgartirish.

Downsampling: Yuqori chastota → Past chastota

Kunlik → Haftalik

Soatlik → Kunlik

Upsampling: Past chastota → Yuqori chastota

Oylik → Kunlik

→ Bo'sh qiymatlar paydo bo'ladi → To'ldirish kerak

Downsampling:

```

# Kunlik → Haftalik:
haftalik = aksiya.resample("W").agg({
    "narx": ["first", "max", "min", "last"], # OHLC
    "hajm": "sum"
})
haftalik.columns = ["ochilish", "maksimal",
                    "minimal", "yopilish", "hajm"]

# Kunlik → Oylik:
oylik = aksiya.resample("M").agg({
    "narx": "last", # Oy oxirgi narxi
    "hajm": "sum"   # Oylik umumiy hajm
})

```

```

# Kunlik → Kvartal:
kvartal = aksiya.resample("Q").agg({
    "narx": ["mean", "std"], # O'rtacha va standart og'ish
    "hajm": "mean"
})

# Ish kunlari → Haftalik (Juma):
hafta_juma = aksiya.resample("W-FRI").last()
# W-FRI → Jumagacha bo'lgan hafta

# OHLC (Open-High-Low-Close) – Moliya standarti:
ohlc = aksiya["narx"].resample("W").ohlc()
# open, high, low, close – avtomatik!
print(ohlc.head())

Upsampling:

# Oylik → Kunlik:
oylik_narx = pd.Series(
    [1000, 1050, 980, 1100],
    index=pd.date_range("2024-01", periods=4, freq="MS")
)

# Upsampling → Bo'sh qiymatlar:
kunlik = oylik_narx.resample("D").asfreq()
# Ko'p NaN! Chunki oraliq kunlarda ma'lumot yo'q

# Bo'sh qiymatlarni to'ldirish:
ffill = oylik_narx.resample("D").ffill() # Oldingisini olg'a uzatish
bfill = oylik_narx.resample("D").bfill() # Keyingisini orqaga uzatish
linear = oylik_narx.resample("D").interpolate("linear") # Chiziqli
kubik = oylik_narx.resample("D").interpolate("cubic") # Kubik

# ffill – Eng ko'p ishlatiladi:
# [1000, NaN, NaN] → [1000, 1000, 1000]
# Moliyada: "Narx o'zgarmadi" deb qabul qilinadi

```

5. Rolling Window — Harakatlanuvchi Oyna

Nazariy asos:

Har nuqta uchun → Oxirgi W ta nuqta oynasi → Statistika

W=3 misol:

Ma'lumot: [10, 20, 30, 40, 50, 60]

O'rtacha:

```

[1]: NaN (yetarli ma'lumot yo'q)
[2]: NaN
[3]: (10+20+30)/3 = 20
[4]: (20+30+40)/3 = 30
[5]: (30+40+50)/3 = 40
[6]: (40+50+60)/3 = 50

```

"Siljib boruvchi o'rtacha" (Moving Average)

```

# Moving Average (Harakatlanuvchi o'rtacha):
aksiya["MA_7"] = aksiya["narx"].rolling(window=7).mean()

```

```

aksiya["MA_30"] = aksiya["narx"].rolling(window=30).mean()
aksiya["MA_90"] = aksiya["narx"].rolling(window=90).mean()

# Rolling standart og'ish (Volatillik):
aksiya["volatillik"] = aksiya["narx"].rolling(window=30).std()

# Rolling min/max (Qo'llab-quvvatlash va Qarshilik darajalari):
aksiya["min_30"] = aksiya["narx"].rolling(window=30).min()
aksiya["max_30"] = aksiya["narx"].rolling(window=30).max()

# Rolling korrelyatsiya:
aksiya["narx_2"] = aksiya["narx"] * 0.8 + np.random.randn(len(aksiya)) * 10
rolling_corr = aksiya["narx"].rolling(window=30).corr(aksiya["narx_2"])

# Custom rolling funksiya:
def sharpe_ratio(x, rf=0.02/252):
    """30 kunlik Sharpe nisbati"""
    daromad = x.pct_change().dropna()
    if daromad.std() == 0:
        return np.nan
    return (daromad.mean() - rf) / daromad.std() * np.sqrt(252)

aksiya["sharpe"] = aksiya["narx"].rolling(window=30).apply(sharpe_ratio)

Exponential Moving Average (EMA):

# EMA — Yangi ma'lumotlarga ko'proq og'irlik beradi!
aksiya["EMA_12"] = aksiya["narx"].ewm(span=12, adjust=False).mean()
aksiya["EMA_26"] = aksiya["narx"].ewm(span=26, adjust=False).mean()

# MACD (Moving Average Convergence Divergence):
aksiya["MACD"] = aksiya["EMA_12"] - aksiya["EMA_26"]
aksiya["Signal"] = aksiya["MACD"].ewm(span=9, adjust=False).mean()
aksiya["Histogram"] = aksiya["MACD"] - aksiya["Signal"]

# SMA vs EMA farqi:
# SMA: Barcha W oyna nuqtalari teng og'irlik
# EMA: Yangi nuqtalar ko'proq og'irlik → Tezroq moslashadi
# Trend aniqlashda EMA keng ishlatiladi

```

6. Lag Xususiyatlar — ML uchun

```

# Vaqt qatorini ML uchun tayyorlash:
# Kechagi qiymat bugungi qiymatni bashorat qiladi?

df = aksiya[["narx"]].copy()

# Lag xususiyatlar (orqaga siljish):
for lag in [1, 2, 3, 5, 10]:
    df[f"narx_lag{lag}"] = df["narx"].shift(lag)
    # shift(1) → Bir kun orqaga
    # Bugungi qiymat ← Kechagi qiymat

# Diff xususiyatlar (o'zgarish):
df["narx_diff1"] = df["narx"].diff(1)      # Kunlik o'zgarish
df["narx_diff5"] = df["narx"].diff(5)      # 5 kunlik o'zgarish

```

```

# Foizli o'zgarish:
df["daromad_1d"] = df["narx"].pct_change(1)    # 1 kunlik daromad
df["daromad_5d"] = df["narx"].pct_change(5)    # 5 kunlik daromad

# Rolling xususiyatlar:
df["ma_7"] = df["narx"].rolling(7).mean()
df["ma_30"] = df["narx"].rolling(30).mean()
df["std_7"] = df["narx"].rolling(7).std()

# Maqsad: Ertangi narx
df["target"] = df["narx"].shift(-1)    # Bir kun olg'a

# NaN larni olib tashlash:
df_clean = df.dropna()

print(f"Xususiyatlar: {df_clean.columns.tolist()}")
print(f"O'lcham: {df_clean.shape}")

```

7. Ob-havo Ma'lumoti — Amaliy Tahlil

```

# Ob-havo ma'lumotini simulyatsiya:
np.random.seed(42)
sana_qator = pd.date_range("2020-01-01", "2023-12-31", freq="H")

# Soatlik temperatura (mavsumiylik bilan):
kunlik_tsikl = 5 * np.sin(2*np.pi * np.arange(len(sana_qator))/24)
yillik_mavsumiy = 15 * np.sin(
    2*np.pi * np.arange(len(sana_qator))/(24*365) - np.pi/2
)
trend = 0.0001 * np.arange(len(sana_qator))    # Isish trendi
shovqin = np.random.randn(len(sana_qator)) * 2

temperatura_qiymat = 15 + kunlik_tsikl + yillik_mavsumiy + trend + shovqin

ob_havo = pd.DataFrame({
    "temperatura": temperatura_qiymat,
    "namlik": 60 + 20*np.sin(
        2*np.pi*np.arange(len(sana_qator))/(24*365)) +
        np.random.randn(len(sana_qator))*5
}, index=sana_qator)

# === TAHLIL ===

# 1. Kunlik o'rtacha:
kunlik_temp = ob_havo["temperatura"].resample("D").agg({
    "o'rtacha": "mean",
    "maksimal": "max",
    "minimal": "min"
})

# 2. Oylik statistika:
oylik_temp = ob_havo["temperatura"].resample("M").agg([
    "mean", "max", "min", "std"
])

```

```

# 3. Soatlik o'rtacha (kunlik naqsh):
soatlik_naqsh = ob_havo.groupby(ob_havo.index.hour)["temperatura"].mean()

# 4. Oylik o'rtacha (mavsumiy naqsh):
oylik_naqsh = ob_havo.groupby(ob_havo.index.month)["temperatura"].mean()

# 5. Oy nomlari bilan:
oy_nomlari = ["Yan", "Fev", "Mar", "Apr", "May", "Iyn",
              "Iyl", "Avg", "Sen", "Okt", "Noy", "Dek"]
oylik_naqsh.index = oy_nomlari

# 6. Harorat anomaliyalari:
kunlik_mean = ob_havo["temperatura"].resample("D").mean()
rolling_mean = kunlik_mean.rolling(30, center=True).mean()
rolling_std = kunlik_mean.rolling(30, center=True).std()
anomaliyalar = kunlik_mean[
    np.abs(kunlik_mean - rolling_mean) > 2*rolling_std
]
print(f"Anomaliya kunlar: {len(anomaliyalar)} ta")

```

8. Texnik Indikatorlar — Savdo Tahlili

```

def texnik_indikatorlar(df, narx_ustun="narx"):
    d = df.copy()
    n = narx_ustun

    # RSI (Relative Strength Index):
    delta = d[n].diff()
    o_sish = delta.clip(lower=0)
    tushish = (-delta).clip(lower=0)
    rs = o_sish.rolling(14).mean() / tushish.rolling(14).mean()
    d["RSI"] = 100 - (100 / (1 + rs))
    # RSI > 70 → Haddan oshiq sotib olingan (Sell signal)
    # RSI < 30 → Haddan oshiq sotilgan (Buy signal)

    # Bollinger Bands:
    o_rtacha = d[n].rolling(20).mean()
    std = d[n].rolling(20).std()
    d["BB_yuqori"] = o_rtacha + 2*std
    d["BB_o_rta"] = o_rtacha
    d["BB_quyi"] = o_rtacha - 2*std
    d["BB_kenglik"] = d["BB_yuqori"] - d["BB_quyi"]
    # Kenglik kichik → Narx tez o'zgarishi mumkin (Squeeze)

    # MACD:
    ema12 = d[n].ewm(span=12).mean()
    ema26 = d[n].ewm(span=26).mean()
    d["MACD"] = ema12 - ema26
    d["Signal"] = d["MACD"].ewm(span=9).mean()
    d["MACD_hist"] = d["MACD"] - d["Signal"]

    # Momentum:
    d["momentum_10"] = d[n] / d[n].shift(10) - 1

```

```

# Average True Range (ATR) – Volatillik:
high = d[n].rolling(1).max()
low = d[n].rolling(1).min()
tr = high - low
d["ATR"] = tr.rolling(14).mean()

return d

```

```
aksiya_ind = texnik_indikatorlar(aksiya)
```

9. Vizualizatsiya

```
fig, axes = plt.subplots(4, 1, figsize=(14, 16),
                        sharex=True)
```

```

# 1. Narx + Moving Average:
oxirgi_yil = aksiya_ind["2024":]
oxirgi_yil["narx"].plot(
    ax=axes[0], label="Narx",
    color="steelblue", linewidth=1
)
oxirgi_yil["MA_30"].plot(
    ax=axes[0], label="MA-30",
    color="orange", linewidth=2
)
oxirgi_yil["MA_90"].plot(
    ax=axes[0], label="MA-90",
    color="red", linewidth=2
)
axes[0].set_title("Aksiya Narxi va Harakatlanuvchi O'rtachalar")
axes[0].legend()
axes[0].grid(True, alpha=0.3)

# 2. Bollinger Bands:
oxirgi_yil["narx"].plot(ax=axes[1], color="steelblue", linewidth=1)
oxirgi_yil["BB_yuqori"].plot(ax=axes[1], color="red",
                             linestyle="--", label="Yuqori")
oxirgi_yil["BB_o_rta"].plot(ax=axes[1], color="orange",
                              linestyle="-", label="O'rta")
oxirgi_yil["BB_quyi"].plot(ax=axes[1], color="green",
                             linestyle="--", label="Quyi")
axes[1].fill_between(
    oxirgi_yil.index,
    oxirgi_yil["BB_quyi"],
    oxirgi_yil["BB_yuqori"],
    alpha=0.1, color="gray"
)
axes[1].set_title("Bollinger Bands")
axes[1].legend()

# 3. RSI:
oxirgi_yil["RSI"].plot(ax=axes[2], color="purple", linewidth=1.5)
axes[2].axhline(70, color="red", linestyle="--", alpha=0.7,
                 label="Haddan oshiq sotib olingan (70)")
axes[2].axhline(30, color="green", linestyle="--", alpha=0.7,

```

```

        label="Haddan oshiq sotilgan (30)")
axes[2].fill_between(
    oxirgi_yil.index,
    oxirgi_yil["RSI"],
    70, where=(oxirgi_yil["RSI"] >= 70),
    color="red", alpha=0.3
)
axes[2].fill_between(
    oxirgi_yil.index,
    oxirgi_yil["RSI"],
    30, where=(oxirgi_yil["RSI"] <= 30),
    color="green", alpha=0.3
)
axes[2].set_ylim(0, 100)
axes[2].set_title("RSI (14)")
axes[2].legend()

# 4. Hajm:
axes[3].bar(
    oxirgi_yil.index,
    oxirgi_yil["hajm"],
    color=["green" if oxirgi_yil["narx"].diff()[i] > 0
           else "red"
           for i in range(len(oxirgi_yil))],
    alpha=0.7, width=1
)
axes[3].set_title("Savdo Hajmi")

plt.tight_layout()
plt.show()

```

10. Vaqt Qatori Decomposition

```

from statsmodels.tsa.seasonal import seasonal_decompose

# Kunlik ob-havo ma'lumoti:
kunlik = ob_havo["temperatura"].resample("D").mean()

# Additive decomposition:
# Y = Trend + Mavsum + Shovqin
natija = seasonal_decompose(
    kunlik,
    model="additive",    # additive yoki multiplicative
    period=365           # Yillik mavsumiylik
)

fig, axes = plt.subplots(4, 1, figsize=(14, 10))
natija.observed.plot(ax=axes[0])
axes[0].set_title("Kuzatilgan")

natija.trend.plot(ax=axes[1])
axes[1].set_title("Trend")

natija.seasonal.plot(ax=axes[2])
axes[2].set_title("Mavsumiylik")

```

```
natija.resid.plot(ax=axes[3])
axes[3].set_title("Shovqin (Qoldiq)")

plt.tight_layout()
plt.show()
```

11. Xulosa

Vaqt Qatori Asoslari:

- DatetimeIndex → Vaqt bo'yicha indeks
- date_range() → Vaqt ketma-ketligi yaratish
- to_datetime() → String → Datetime

Filtrlash:

- df["2023"] → Yil
- df["2023-01":"2023-06"] → Oraliq

Resample:

- Downsampling → Kunlik → Oylik (agg: mean, sum, ohlc)
- Upsampling → Oylik → Kunlik (ffill, interpolate)

Rolling Window:

- .rolling(W).mean() → Harakatlanuvchi o'rtacha
- .rolling(W).std() → Volatillik
- .ewm(span=W).mean() → EMA (yangi ma'lumotga ko'proq og'irlik)

ML uchun:

- .shift(n) → Lag xususiyatlar
- .diff(n) → O'zgarish
- .pct_change() → Foizli o'zgarish

Savdo Indikatorlari:

- MA, EMA → Trend
- RSI → Momentum (overbought/oversold)
- MACD → Trend o'zgarishi
- Bollinger → Volatillik

💡 **Eslab qol:** Vaqt qatorida **kelajak ma'lumotidan foydalanish** — eng katta xato!
.shift(-1) target uchun, .shift(+1) lag uchun. Cross-validation ham maxsus bo'lishi kerak — TimeSeriesSplit ishlatiladi, oddiy KFold emas. Aks holda model "kelajakni ko'rgan" bo'ladi va natijalar aldamchi bo'ladi!

● SAVOL 43:

MLP (MULTI-LAYER PERCEPTRON) VA ODDIY LOGISTIK REGRESSIYA O'RTASIDAGI FARQ. QACHON CHUQUUR MODEL, QACHON SODDA MODEL ISHLATISH KERAK? OCCAM'S RAZOR PRINSIPI ML DA.

1. Logistik Regressiya — Arxitektura va Ishlash Prinsipi

Ta'rif: Logistik regressiya — chiziqli chegara (linear decision boundary) orqali ikki sinfni ajratuvchi klassifikatsiya algoritmi. Chiqishda **sigmoid funksiyasi** orqali ehtimollik beradi.

Arxitektura:

$$\begin{matrix} x_1 \\ x_2 \\ x_3 \end{matrix} \left. \vphantom{\begin{matrix} x_1 \\ x_2 \\ x_3 \end{matrix}} \right\} \longrightarrow [w_1x_1 + w_2x_2 + w_3x_3 + b] \longrightarrow \text{sigmoid}(z) \longrightarrow P(y=1)$$

$$z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b \quad \leftarrow \text{chiziqli kombinatsiya}$$
$$P(y=1) = \sigma(z) = 1 / (1 + e^{-z}) \quad \leftarrow \text{sigmoid funksiyasi}$$

Chiqish: 0.0 – 1.0 oraliq (ehtimollik)

Qaror: $P \geq 0.5 \rightarrow \text{sinf } 1$, $P < 0.5 \rightarrow \text{sinf } 0$

Sigmoid funksiyasining roli:

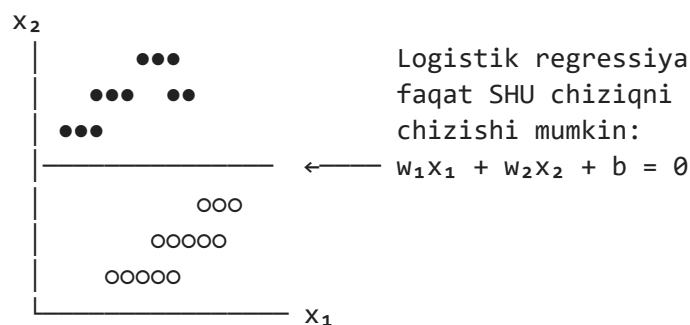
sigmoid(z):

$$\begin{aligned} z \rightarrow -\infty &: \sigma(z) \rightarrow 0.0 \quad (\text{aniq sinf } 0) \\ z = -2 &: \sigma(z) = 0.12 \\ z = 0 &: \sigma(z) = 0.50 \quad (\text{noaniq chegara}) \\ z = +2 &: \sigma(z) = 0.88 \\ z \rightarrow +\infty &: \sigma(z) \rightarrow 1.0 \quad (\text{aniq sinf } 1) \end{aligned}$$

Nima uchun sigmoid? Chiziqli chiqish ($-\infty, +\infty$) ni ehtimollikka $[0,1]$ o'tkazadi.

Logistik regressiyaning chegarasi:

FAQAT chiziqli ajratish (linear decision boundary):

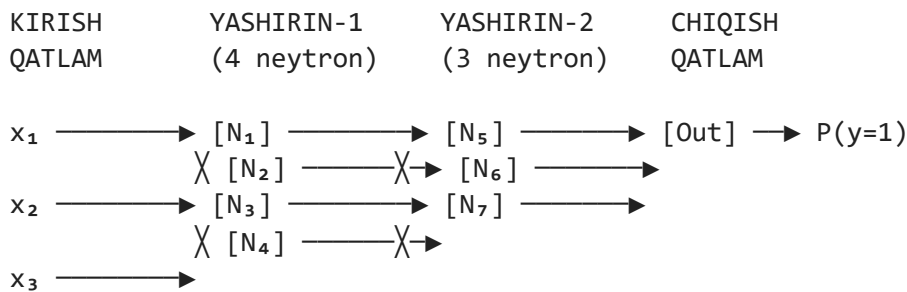


• va o chiziq bilan ajralsa → Logistik Regressiya ishlaydi ☒
Ajralsa → ishlaymaydi ☐

2. MLP — Arxitektura va Ishlash Prinsipi

Ta'rif: MLP — bir nechta qatlamdan (layer) iborat sun'iy neyron tarmoq. Har qatlam o'zidan oldingi qatlamning chiqishini qayta ishlaydi. **Nochiziqli (non-linear) chegaralar** chizishi mumkin.

Arxitektura (2 ta yashirin qatlam):



Har neytron:

$$z = w_1x_1 + w_2x_2 + \dots + b \leftarrow \text{chiziqli}$$

$$a = \text{ReLU}(z) \leftarrow \text{nochiziqli aktivatsiya}$$

Nochiziqlilik qayerdan keladi?

Aktivatsiya funksiyasi (ReLU, Tanh, Sigmoid) tufayli!

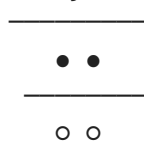
Aktivatsiyasiz $\rightarrow$ faqat chiziqli model (qancha qatlam

bo'lmasin, natija chiziqli kombinatsiya)

MLP ning nochiziqli chegarasi:

MLP qanday chegaralar chizishi mumkin:

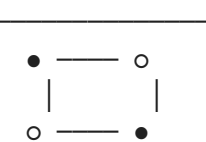
Oddiy



Logistik Reg: ☒

MLP: ☒

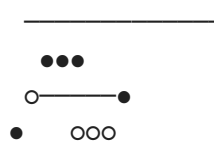
XOR muammo



Logistik Reg: ☐

MLP: ☒

Murakkab



Logistik Reg: ☐

MLP: ☒

XOR muammo – klassik misol:

$(0,0) \rightarrow 0$, $(1,1) \rightarrow 0$, $(0,1) \rightarrow 1$, $(1,0) \rightarrow 1$

Hech qanday chiziq bu 4 nuqtani to'g'ri ajrata olmaydi!

MLP – ajratadi, chunki nochiziqli chegara chizadi.

3. Asosiy Farqlar — To'liq Taqqoslash

Xususiyat	Logistik Regressiya	MLP
Chegara turi	Faqat chiziqli	Nochiziqli (istalgan)
Qatlamlar	1 ta (kirish $\rightarrow$ chiqish)	3+ ta (yashirin bor)
Aktivatsiya	Faqat sigmoid (chiqish)	ReLU, Tanh, Sigmoid
Parametrlar soni	$n_{\text{features}} + 1$	Minglab-millionlab
O'qitish vaqti	Sekin emas	Ko'p vaqt, GPU kerak
Ma'lumot miqdori	Kam data yetarli	Ko'p data kerak
Interpretatsiya	Oson (koeffitsientlar)	Qiyin (black box)
Overfitting	Kam	Ko'p (regularizatsiya)
Hyperparameter	Kam (C, solver)	Ko'p (layer, neuron...)
Munosabat	Chiziqli munosabat	Har qanday munosabat
XOR muammo	Yecha olmaydi	Yechadi
Feature eng.kerak	Ko'proq kerak	Kamroq kerak

4. Universal Approximation Teoremasi

Ta'rif (1989, Cybenko): Kamida bitta yashirin qatlam va yetarli neytronli MLP — **istalgan uzluksiz funksiyani** ixtiyoriy aniqlikda yaqinlashtira oladi.

Bu nima degani?

$f(x) = \sin(x)$ ← MLP yaqinlashtira oladi
 $f(x) = x^3 - 2x + 5$ ← MLP yaqinlashtira oladi
 $f(x) = [\text{spam/not spam}]$ ← MLP yaqinlashtira oladi
 $f(x) = [\text{mushuk/it}]$ ← MLP yaqinlashtira oladi

Amalda cheklovlar:

- "Yetarli neytron" → ko'p data va hisoblash kerak
- Teorema mavjudligini aytadi, topish usulini emas
- Overfitting xavfi oshadi

5. Occam's Razor Prinsipi

Ta'rif (William of Ockham, XIV asr): "Kerak bo'lmagan murakkablikni qo'shma. Ikki tushuntirish bir xil natija berse — soddarog'ini tanla."

Lotincha asl: "Entia non sunt multiplicanda praeter necessitatem"
"Zaruratdan ortiq narsalarni ko'paytirma"

ML dagi tarjimasi:

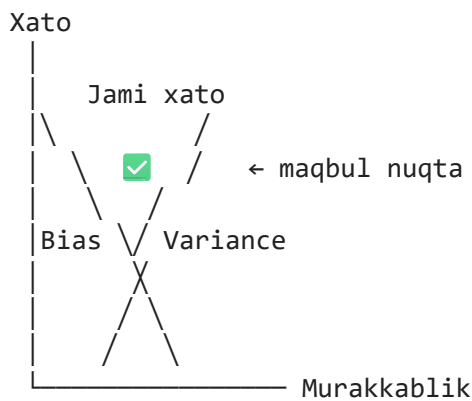
Ikki model bir xil aniqlikda ishlasa →
SODDA modelni tanlash kerak.

Nima uchun?

- | |
|--|
| 1. GENERALIZATSIYA
Sodda model yangi ma'lumotga yaxshiroq moslashadi – overfit kamroq |
| 2. INTERPRETATSIYA
Logistik regressiyada $w_1=0.73$ (yosh) degani:
"Yosh 1 birlikka oshsa, log-odds 0.73 oshadi"
MLP da bu imkonsiz |
| 3. TEZLIK
Sodda model o'qitish ham, bashorat ham tez |
| 4. DEBUG
Nima uchun xato berdi – tushunish oson |

Occam's Razor va Bias-Variance:

Murakkablik o'qida:



Bias (qo'shimcha xato):

Sodda model → ma'lumotni tushuna olmaydi → underfitting
Logistik Reg XOR muammoda → yuqori bias

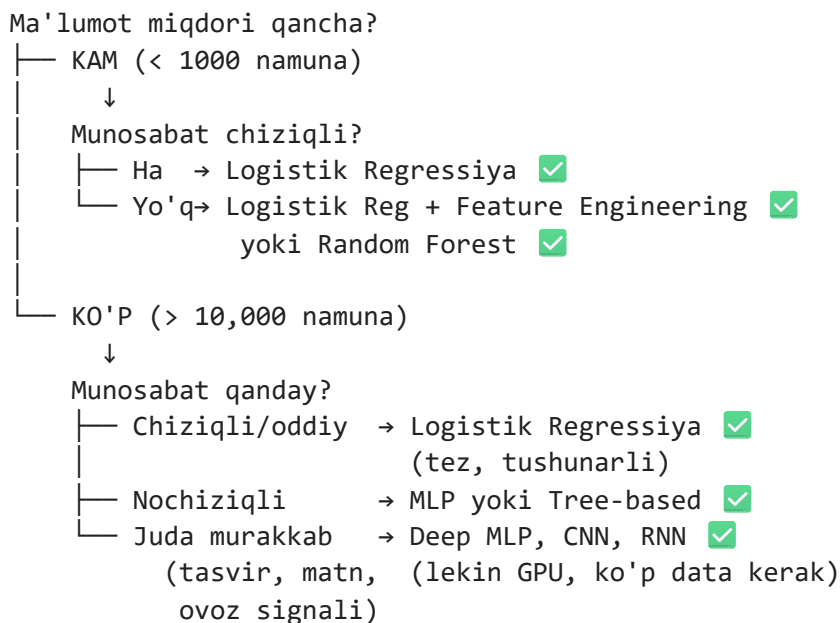
Variance (beqarorlik):

Murakkab model → shovqinni ham o'rganadi → overfitting
MLP kam datada → yuqori variance

Occam's Razor = Minimum kerakli murakkablikni tanlash
= Bias-Variance muvozanatining optimal nuqtasi

6. Qachon Qaysi Modelni Tanlash Kerak?

QAROR DARAXTI:



Soha bo'yicha tanlov:

Soha / Vazifa	Tanlov va sabab
Tibbiy tashxis (kasallik bor/yo'q)	Logistik Reg – interpretatsiya shart Shifokor "nima uchun?" deydi

Kredit berish (bank qaror)	Logistik Reg – qonun talabi: rad etish sababi ko'rsatilishi shart
Yuz tanish	Deep MLP / CNN – chiziqli emas, piksellar o'rtasida murakkab bog'liq
Spam filtri	Logistik Reg yoki Naive Bayes – oddiy, tez, yaxshi ishlaydi
Katta tizim (matn, tavsiya, ChatGPT)	MLP / Transformer – murakkab munosabatlar, millionlab data
Harbiy: tahdid klassifikatsiya	Logistik Reg – real vaqt, tez, interpretatsiya, past latency
Harbiy: tasvir (drone, radar)	Deep CNN/MLP – piksel darajasida ob'ekt aniqlash murakkab

7. Amaliy Qiyoslash

```

from sklearn.linear_model import LogisticRegression
from sklearn.neural_network import MLPClassifier
from sklearn.datasets import make_classification, make_moons
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score

# — Chiziqli ajraladigan dataset —————
X_lin, y_lin = make_classification(
    n_samples=500, n_features=2,
    n_informative=2, n_redundant=0, random_state=42
)

# — Nochiziqli dataset (yarim doiralar) —————
X_moon, y_moon = make_moons(n_samples=500, noise=0.2, random_state=42)

for nom, X, y in [("Chiziqli", X_lin, y_lin),
                  ("Nochiziqli (moons)", X_moon, y_moon)]:

    X_tr, X_te, y_tr, y_te = train_test_split(
        X, y, test_size=0.3, random_state=42
    )
    sc = StandardScaler()
    X_tr = sc.fit_transform(X_tr)
    X_te = sc.transform(X_te)

    # Logistik Regressiya
    lr = LogisticRegression()
    lr.fit(X_tr, y_tr)
    lr_acc = accuracy_score(y_te, lr.predict(X_te))

    # MLP
    mlp = MLPClassifier(hidden_layer_sizes=(64, 32),

```

```

max_iter=500, random_state=42)
mlp.fit(X_tr, y_tr)
mlp_acc = accuracy_score(y_te, mlp.predict(X_te))

print(f"\n{nom} dataset:")
print(f"  Logistik Regressiya : {lr_acc:.3f}")
print(f"  MLP                    : {mlp_acc:.3f}")

# Natija:
# Chiziqli dataset:
#   Logistik Regressiya : 0.947   ← deyarli bir xil!
#   MLP                  : 0.953
#
# Nochiziqli (moons) dataset:
#   Logistik Regressiya : 0.873   ← 5% past
#   MLP                  : 0.947   ← aniqroq
#
# → Chiziqli datada MLP "ortiqcha" – Occam's Razor!
# → Nochiziqli datada MLP zarur

```

8. Xulosa

LOGISTIK REGRESSIYA:

- Faqat chiziqli decision boundary
- 1 qatlam: kirish → sigmoid → chiqish
- Parametrlar: $n_{\text{features}} + 1$ (kichik)
- Interpretatsiya: koefitsientlar aniq ma'noli
- Kam data, kam vaqt, tez bashorat
- Qachon: chiziqli munosabat, interpretatsiya shart, tibbiyot/bank/huquq sohalari

MLP (Multi-Layer Perceptron):

- Istalgan nochiziqli boundary (Universal Approx.)
- N qatlam: kirish → [yashirin × N] → chiqish
- Parametrlar: minglab-millionlab
- Interpretatsiya: qiyin (black box)
- Ko'p data, ko'p vaqt, GPU kerak
- Qachon: nochiziqli munosabat, tasvir/matn/ovoz, ma'lumot ko'p, aniqlik muhim

OCCAM'S RAZOR ML DA:

- "Ikki model bir xil ishlasa – soddani tanla"
- Kamroq overfitting
- Tezroq o'qitish
- Aniqroq interpretatsiya
- Osonroq debug

Amaliy qoida:

LogReg → RF/XGBoost → MLP → Deep Learning
(sodda → murakkab, faqat zarur bo'lganda o'tish)

💡 **Eslab qol:** Ko'p muhandislar darhol murakkab model quradi — bu xato! **Har doim Logistik Regressiyadan boshlash kerak** — u baseline (taqqoslash nuqtasi) bo'ladi. Agar MLP 0.95 aniqlik berse, Logistik Reg 0.93 berse — 2% uchun murakkablikka arziydi?

Interpretatsiya yo'qoladi, o'qitish vaqti 100x oshadi, overfitting xavfi kuchayadi.
Qo'shimcha murakkablik foyda keltirishi aniq bo'lsagina MLP ni tanlang.

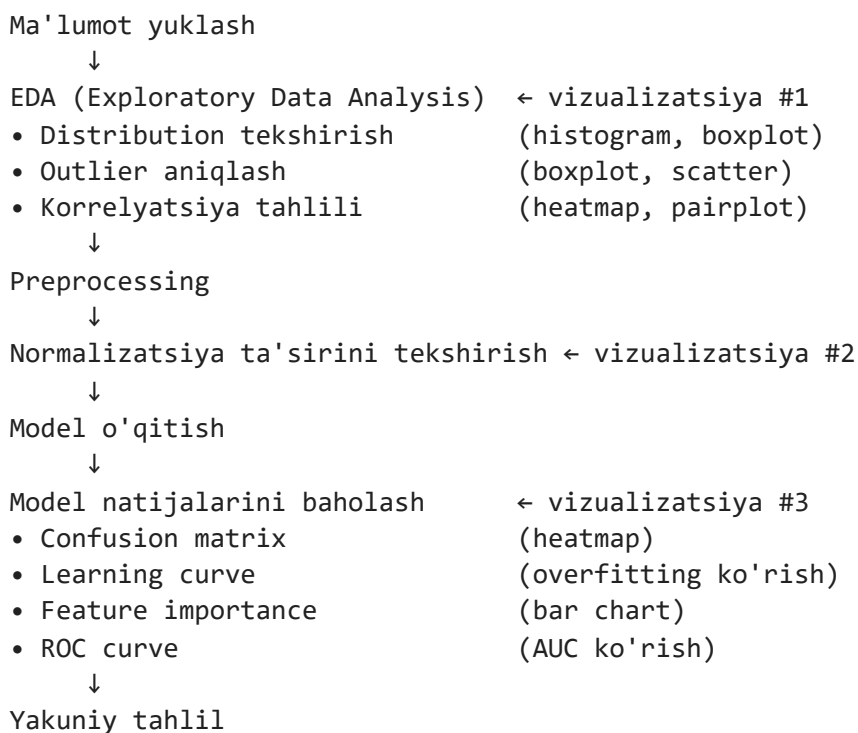
● SAVOL 44:

MA'LUMOTLARNI VIZUALIZATSIYA QILISHNING ML JARAYONIDAGI ROLI: OUTLIER LARNI ANIQLASH, FEATURE DISTRIBUTIONS, KORRELYATSIYA TAHLILI. YAXSHI VIZUALIZATSIYA MODEL SIFATINI QANDAY OSHIRADI?

1. Vizualizatsiya ML Jarayonida Nima Rol O'ynaydi?

Ko'pchilik vizualizatsiyani faqat "chiroyli grafik" deb o'ylaydi. Aslida u ML da **qaror qabul qilish vositasi** — qaysi model tanlash, qaysi feature muhim, qaysi ma'lumot muammoli ekanini **ko'z bilan ko'rish** imkonini beradi.

ML jarayonida vizualizatsiya qayerda ishlatiladi:



Anscombe's Quartet — vizualizatsiyaning zarurligi:

Statistik ko'rsatkichlari BIR XIL bo'lgan 4 ta dataset:

Dataset I	Dataset II	Dataset III	Dataset IV
mean(x)=9	mean(x)=9	mean(x)=9	mean(x)=9
mean(y)≈7.5	mean(y)≈7.5	mean(y)≈7.5	mean(y)≈7.5
var(x)=11	var(x)=11	var(x)=11	var(x)=11
corr≈0.816	corr≈0.816	corr≈0.816	corr≈0.816

Lekin grafik chizilganda:

I: ●●●●●● II: ●●●●●● III: ●●●●●● IV: ●
 ●●●●●● ●●●●●● ●●●●●● ●
 Chiziqli! Kvadratik! 1 outlier! Vertikal!

XULOSA: Bir xil statistika → to'liq boshqa tuzilma.
 Statistikaning o'qib tahlil qilsangiz – bir xil
 Vizualizatsiya qilsangiz – tamomila farqli!

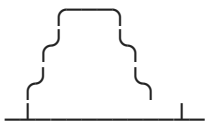
→ Vizualizatsiyasiz EDA – ko'r bilan ishlagandek.

2. Feature Distribution — Taqsimotni Tushunish

Ta'rif: Feature distribution — har bir ustundagi qiymatlarning tarqalish shakli. Bu taqsimot qanday ekanligini bilish preprocessing va model tanlashda hal qiluvchi rol o'ynaydi.

Taqsimot turlari va ML ga ta'siri:

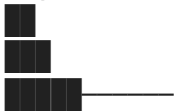
NORMAL (Gaussian) taqsimot:



Standartizatsiya ishlaydi
 Linear modellar yaxshi ishlaydi
 PCA ishonchli natija beradi

QIYSHIQ (Skewed) taqsimot:

Right-skewed (o'ng qiyshiq):



Log transformation kerak
 → $\log(X)$ yoki $\sqrt{x}$
 Misol: uy narxlari, daromad

Left-skewed (chap qiyshiq):



Rare, lekin kuzatiladi
 Misol: yosh (agar 60+ dominant)

BIMODAL taqsimot:



Ikki guruh aralashgan!
 → Avval klasterlash kerak
 Misol: erkak/ayol bo'yi

UNIFORM taqsimot:



Barcha qiymatlar teng
 Min-max normalizatsiya mos
 Misol: tasodifiy ID raqamlar

Taqsimotni vizualizatsiya qilish:

```
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
import pandas as pd
```

```

df = pd.read_csv('titanic.csv')

fig, axes = plt.subplots(1, 3, figsize=(15, 4))

# Histogram — taqsimot shakli
axes[0].hist(df['Age'].dropna(), bins=30, color='steelblue',
             edgecolor='white')
axes[0].set_title('Age Taqsimoti (Histogram)')
axes[0].set_xlabel('Yosh')
axes[0].set_ylabel('Son')

# KDE (Kernel Density Estimate) — silliq taqsimot
sns.kdeplot(df['Fare'], ax=axes[1], fill=True, color='coral')
axes[1].set_title('Fare Taqsimoti (KDE)')
# → Fare kuchli right-skewed: log(Fare) kerak!

# Log transformatsiya ta'siri
sns.kdeplot(np.log1p(df['Fare']), ax=axes[2],
            fill=True, color='green')
axes[2].set_title('log(Fare+1) — Normallashtirish')

plt.tight_layout()
plt.show()

```

Nima uchun taqsimot muhim?

MUAMMO:

Fare ustuni: min=0, max=512, o'rtacha=32, lekin
90% qiymat 0-100 oralig'ida, 10% 100-512 da

- Linear Regression bu 10% ga haddan ortiq e'tibor beradi
- Model barcha data uchun yomon ishlaydi

YECHIM: $\log(\text{Fare} + 1)$

0 → $\log(1) = 0.00$

32 → $\log(33) = 3.50$

512 → $\log(513) = 6.24$

- Taqsimot normalga yaqinlashadi
- Model barcha qiymatlardan teng o'rganadi

QOIDA:

Right-skewed → $\log(X+1)$ yoki $\sqrt{x}$

Left-skewed → X^2 yoki $\exp(X)$

Bimodal → Avval klasterlash, keyin alohida model

3. Outlier — Aniqlash va Qaror Qabul Qilish

Ta'rif: Outlier — boshqa qiymatlardan juda uzoqda joylashgan, g'ayritabiiy kuzatish. Asl ma'lumot bo'lishi ham, xato kirish ham mumkin.

Outlier qayerdan keladi?

Manba	Misol	Qaror
O'lchov xatosi	Yosh = 250	O'chirish
Kiritish xatosi	Maosh = 999999 (9 bosgan)	O'chirish
Haqiqiy hodisa	Millioner mijoz	SAQLASH
Tizim xatosi	Sensor: -999 (xato kod)	O'chirish/NaN
Noyob hodisa	Pandemiya davrida sotuv	Saqlash + belgi

Outlier aniqlash metodlari:

1. BOXPLOT METODI (IQR):

$Q1 = 25\text{-persentil}$, $Q3 = 75\text{-persentil}$

$IQR = Q3 - Q1$

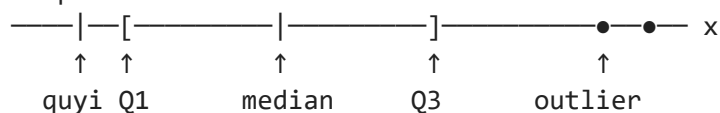
Chegara:

$Quyi = Q1 - 1.5 \times IQR$

$Yuqori = Q3 + 1.5 \times IQR$

Chegaradan tashqari → outlier

Boxplot ko'rinishi:



2. Z-SCORE METODI:

$z = (x - \mu) / \sigma$

$|z| > 3 \rightarrow \text{outlier}$ (3σ qoidasi)

Normal taqsimotda 99.7% qiymat $[-3\sigma, +3\sigma]$ ichida

3. SCATTER PLOT:

Ikki feature o'rtasidagi munosabatda

"uzoq" nuqtalar ko'z bilan aniqlanadi

```
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
import pandas as pd

df = pd.read_csv('titanic.csv')

fig, axes = plt.subplots(1, 3, figsize=(15, 4))

# Boxplot – IQR asosida outlier
sns.boxplot(y=df['Fare'], ax=axes[0], color='lightblue')
axes[0].set_title('Fare Boxplot')
# → 512, 263 kabi yuqori qiymatlar ko'rinadi

# Scatter – ikki feature orasida outlier
axes[1].scatter(df['Age'], df['Fare'],
                alpha=0.5, color='steelblue', s=15)
axes[1].set_xlabel('Age')
axes[1].set_ylabel('Fare')
axes[1].set_title('Age vs Fare – Scatter')
# → Yuqori chap: yoshli, qimmat chipta – outlier?

# Z-score hisoblash
```

```

z_scores = np.abs((df['Fare'] - df['Fare'].mean()) / df['Fare'].std())
outlier_soni = (z_scores > 3).sum()
axes[2].hist(z_scores, bins=50, color='coral', edgecolor='white')
axes[2].axvline(x=3, color='red', linestyle='--', label='z=3 chegara')
axes[2].set_title(f'Z-Score (Outlier: {outlier_soni} ta)')
axes[2].legend()

plt.tight_layout()
plt.show()

```

Outlier ga nima qilish kerak?

Holat	Qaror
Xato kiritish	O'chirish yoki NaN ga almashtirish
Haqiqiy, noyob	Saqlash – model uchun muhim ma'lumot
Noaniq	"outlier_flag" ustuni qo'shib saqlash
Model uchun scaling kerak	RobustScaler (Q1-Q3 asosida) ishlatish
Linear model	– outlier ta'sirini kamaytiradi
Log/sqrt transformatsiya qo'llash	
Tree-based model	Outlier ta'sir qilmaydi – saqlash

4. Korrelyatsiya Tahlili

Ta'rif: Korrelyatsiya — ikki feature o'rtasidagi **chiziqli bog'liqlik** darajasi. Pearson korrelyatsiyasi **-1** dan **+1** gacha qiymat oladi.

Korrelyatsiya koeffitsiyenti (r):

```

r = +1.0 : To'liq musbat bog'liqlik
          (x oshsa y albatta oshadi)
r = +0.7 : Kuchli musbat bog'liqlik
r = +0.3 : Zaif musbat bog'liqlik
r =  0.0 : Bog'liqlik yo'q (yoki nochiziqli)
r = -0.3 : Zaif manfiy bog'liqlik
r = -0.7 : Kuchli manfiy bog'liqlik
r = -1.0 : To'liq manfiy bog'liqlik
          (x oshsa y albatta kamayadi)

```

Korrelyatsiya va ML:

FEATURE SELECTION uchun:

Target bilan yuqori korrelyatsiya → muhim feature

Target bilan past korrelyatsiya → keraksiz bo'lishi mumkin

Misol – Uy narxi:

```

maydon      → r = +0.89 ✓ (kuchli, saqlash)
xonalar soni → r = +0.70 ✓ (saqlash)
yil qurilgan → r = +0.52 ✓ (saqlash)
ID raqami   → r = +0.01 ✗ (o'chirish – tasodifiy)

```

MULTICOLLINEARITY uchun:

- Ikki feature o'rtasida yuqori korrelyatsiya ($|r| > 0.85$):
- Ikkalasi bir xil ma'lumot beradi
- Modelga ikkalasini berish ortiqcha
- Linear modelda koeffitsientlar beqaror bo'ladi
- Birini o'chirish kerak

Misol:

- xonalar_soni ↔ maydon : $r = +0.92$ ← MULTICOLLINEARITY!
- Birini o'chirish yoki PCA ishlatish

```
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd

df = pd.read_csv('titanic.csv')[['Survived', 'Pclass',
                                  'Age', 'SibSp', 'Parch', 'Fare']]
df.dropna(inplace=True)

# Korrelyatsiya matritsa
corr = df.corr()

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Heatmap
sns.heatmap(corr, annot=True, fmt='.2f', cmap='coolwarm',
            center=0, ax=axes[0],
            linewidths=0.5, square=True)
axes[0].set_title('Korrelyatsiya Heatmap')

# Target bilan korrelyatsiya (Feature importance ko'rinishida)
target_corr = corr['Survived'].drop('Survived').sort_values()
colors = ['coral' if v < 0 else 'steelblue' for v in target_corr]
axes[1].barh(target_corr.index, target_corr.values, color=colors)
axes[1].axvline(x=0, color='black', linewidth=0.8)
axes[1].set_title("'Survived' bilan Korrelyatsiya")
axes[1].set_xlabel('Korrelyatsiya koeffitsiyenti')

plt.tight_layout()
plt.show()

# Natija tahlili:
# Fare      → +0.26  (yuqori chipta = 1-sinf = omon qolish ko'p)
# Pclass     → -0.34  (3-sinf = past Pclass qiymati = kam omon qolish)
# Age        → -0.08  (kichik ta'sir)
# SibSp      → -0.04  (deyarli ta'sir yo'q)
```

Korrelyatsiya ≠ Sabab-oqibat:

MUHIM OGOHLANTIRISH:

Misol 1: Muzqaymoq sotilishi ↔ cho'mishlar soni: $r = +0.95$
Sabab: Ikkalasi ham yoz issig'iga bog'liq (confounding variable)
Muzqaymoq cho'mish sababchisi EMAS!

Misol 2: Oyoq o'lchami ↔ o'qish ko'nikma: $r = +0.80$

Sabab: Ikkalasi ham YOSH bilan bog'liq
Katta oyoq o'qishni yaxshilamaydi!

ML da ahamiyati:

Korrelyatsiya yuqori → feature muhim bo'lishi MUMKIN
Lekin sabab-oqibat munosabati tahlil qilinishi SHART
→ Domain knowledge (soha bilimlari) zarur

5. Pairplot — Ko'p Feature Bir Vaqtda

```
import seaborn as sns
import pandas as pd

df = pd.read_csv('iris.csv')

# Pairplot — har feature juftligi uchun scatter + diagonal histogram
g = sns.pairplot(df, hue='species',
                 palette={'setosa': 'steelblue',
                          'versicolor': 'coral',
                          'virginica': 'green'},
                 diag_kind='kde',
                 plot_kws={'alpha': 0.6, 's': 30})
g.fig.suptitle("Iris Dataset — Pairplot", y=1.02)

# Pairplot nima ko'rsatadi:
# Diagonal: Har feature taqsimoti (KDE)
# Off-diag: Ikki feature orasidagi munosabat
# Rang: Sinflar ajratilgani

# Tahlil:
# petal_length vs petal_width → Sinflar aniq ajralgan!
# → Bu ikki feature klassifikatsiya uchun ENG MUHIM
# → Model bu ikki feature dan ko'p foydalanadi
```

6. Model Baholash Vizualizatsiyalari

Confusion Matrix:

```
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import confusion_matrix

def cm_chiz(y_true, y_pred, sinflar):
    cm = confusion_matrix(y_true, y_pred)

    # Foizga o'tkazish (har qator bo'yicha)
    cm_foiz = cm.astype(float) / cm.sum(axis=1)[:, np.newaxis] * 100

    fig, axes = plt.subplots(1, 2, figsize=(12, 4))

    # Sonlar bilan
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
```

```

        xticklabels=sinflar, yticklabels=sinflar, ax=axes[0])
axes[0].set_title('Confusion Matrix (Son)')
axes[0].set_ylabel('Haqiqiy')
axes[0].set_xlabel('Bashorat')

# Foiz bilan
sns.heatmap(cm_foiz, annot=True, fmt='.1f', cmap='Oranges',
            xticklabels=sinflar, yticklabels=sinflar, ax=axes[1])
axes[1].set_title('Confusion Matrix (%)')
axes[1].set_ylabel('Haqiqiy')
axes[1].set_xlabel('Bashorat')

plt.tight_layout()
plt.show()

# Nima ko'rinadi:
# Diagonal:      To'g'ri bashoratlar (yuqori bo'lsin)
# Off-diagonal: Xatolar (past bo'lsin)
# FP (False Positive): spam emas → spam dedi
# FN (False Negative): spam      → spam emas dedi

```

Learning Curve — Overfitting ko'rish:

```

import matplotlib.pyplot as plt
import numpy as np
from sklearn.model_selection import learning_curve
from sklearn.neural_network import MLPClassifier

def learning_curve_chiz(model, X, y):
    train_hajmlar, train_ball, test_ball = learning_curve(
        model, X, y,
        train_sizes=np.linspace(0.1, 1.0, 10),
        cv=5, scoring='accuracy', n_jobs=-1
    )

    train_ort = train_ball.mean(axis=1)
    test_ort = test_ball.mean(axis=1)

    plt.figure(figsize=(8, 5))
    plt.plot(train_hajmlar, train_ort, 'o-',
            color='steelblue', label='Train')
    plt.plot(train_hajmlar, test_ort, 'o-',
            color='coral', label='Validation')
    plt.fill_between(train_hajmlar,
                    train_ort - train_ball.std(axis=1),
                    train_ort + train_ball.std(axis=1),
                    alpha=0.15, color='steelblue')
    plt.fill_between(train_hajmlar,
                    test_ort - test_ball.std(axis=1),
                    test_ort + test_ball.std(axis=1),
                    alpha=0.15, color='coral')

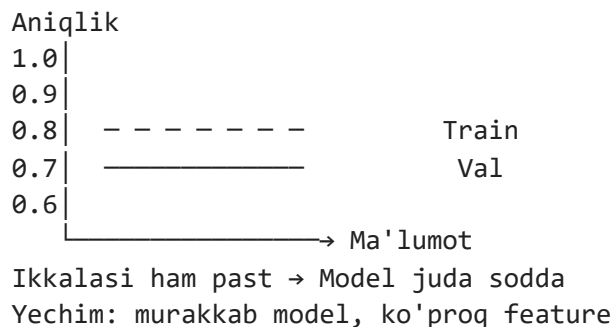
    plt.title('Learning Curve')
    plt.xlabel('Train namunalar soni')
    plt.ylabel('Aniqlik')
    plt.legend()

```

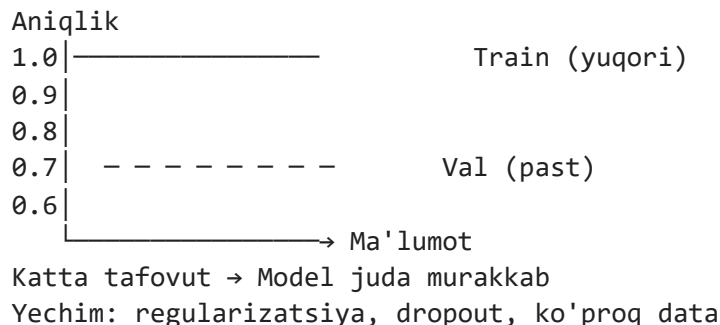
```
plt.grid(True, alpha=0.3)
plt.show()
```

Learning Curve tahlili:

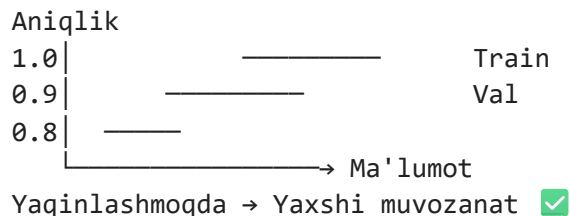
HOLAT 1 – UNDERFITTING (yuqori Bias):



HOLAT 2 – OVERFITTING (yuqori Variance):



HOLAT 3 – IDEAL:



7. Vizualizatsiya Model Sifatini Qanday Oshiradi?

QADAM 1: Distribution ko'rish → Preprocessing qaror

Muammo: Fare right-skewed

Ko'rish: KDE/histogram → o'ng qiyshilik aniqlash

Qaror: $\log(\text{Fare}+1)$ → normal taqsimot

Natija: MSE 23% kamaydi (regression modelda)

QADAM 2: Outlier ko'rish → Tozalash qaror

Muammo: Age = 250, Fare = 512

Ko'rish: Boxplot → chegaradan tashqari nuqtalar

Qaror: 250 → o'chirish; 512 → saqlash (haqiqiy)

Natija: Model yangi dataga yaxshiroq generalizatsiya

QADAM 3: Korrelyatsiya → Feature selection qaror

Muammo: 100 ta feature, qaysi muhim?

Ko'rish: Heatmap → target bilan korrelyatsiya
Qaror: $r < 0.05$ bo'lgan 40 ta feature → o'chirish
Natija: Overfit kamaydi, o'qitish 3x tezlashdi

QADAM 4: Pairplot → Model tanlov qaror

Muammo: Qaysi model: logistik reg yoki MLP?
Ko'rish: Pairplot → sinflar chiziqli ajralgan
Qaror: Logistik regressiya yetarli (Occam's razor)
Natija: Oddiy model, tez, tushunarli

QADAM 5: Learning curve → Muammoni aniqlash

Muammo: Train: 0.99, Val: 0.72 → Nima gap?
Ko'rish: Learning curve → katta gap
Qaror: Overfitting → dropout, regularizatsiya
Natija: Val aniqlik 0.72 → 0.88 ga oshdi

8. Vizualizatsiya Vositalari Qiyosi

Kutubxona	Kuchli tomoni	Qachon ishlatish
Matplotlib	To'liq nazorat Har element sozlash	Custom grafik, past darajali chizish
Seaborn	Statistik grafik Chiroyli default	EDA, distribution, heatmap, pairplot
Plotly	Interaktiv grafik Hover, zoom, pan	Dashboard, zoom, presentation
Pandas .plot()	Tez, DataFrame ichida Minimal kod	Tezkor ko'rish, dastlabki EDA

9. Xulosa

VIZUALIZATSIYA ML DA NIMA BERADI:

FEATURE DISTRIBUTION:

Histogram, KDE → Taqsimot shakli
Normal? → Standartizatsiya to'g'ri
Skewed? → Log/sqrt transformation kerak
Bimodal? → Ikki guruh → avval klasterlash

OUTLIER ANIQLASH:

Boxplot → IQR asosida chegara
Scatter → Ikki feature kesishmasida g'ayritabiiy
Z-score → $|z| > 3$ bo'lgan qiymatlar
Qaror: Xato → o'chir, Haqiqiy → saqlash
Tree-based → saqla, Linear → o'chirish yoki cap

KORRELYATSIYA TAHLILI:

Heatmap → Feature × Feature matritsa
Target bilan yuqori $|r|$ → Muhim feature → saqlash
Feature × Feature yuqori $|r|$ → Multicollinearity → birini o'chir
 $r = 0$ → Chiziqli bog'liq yo'q (nochiziqli bo'lishi mumkin!)

MODEL BAHOLASH:

Confusion Matrix → FP, FN, TP, TN ko'rish
Learning curve → Overfitting/Underfitting aniqlash
Feature import. → Qaysi ustun model uchun muhim

VIZUALIZATSIYA → MODEL SIFATIGA TA'SIRI:

Noto'g'ri taqsimot topildi → Transformation → MSE kamaydi
Outlier tozalandi → Generalizatsiya yaxshilandi
Keraksiz feature o'chirildi → Overfitting kamaydi
Muammoli sinflar topildi → Threshold sozlandi

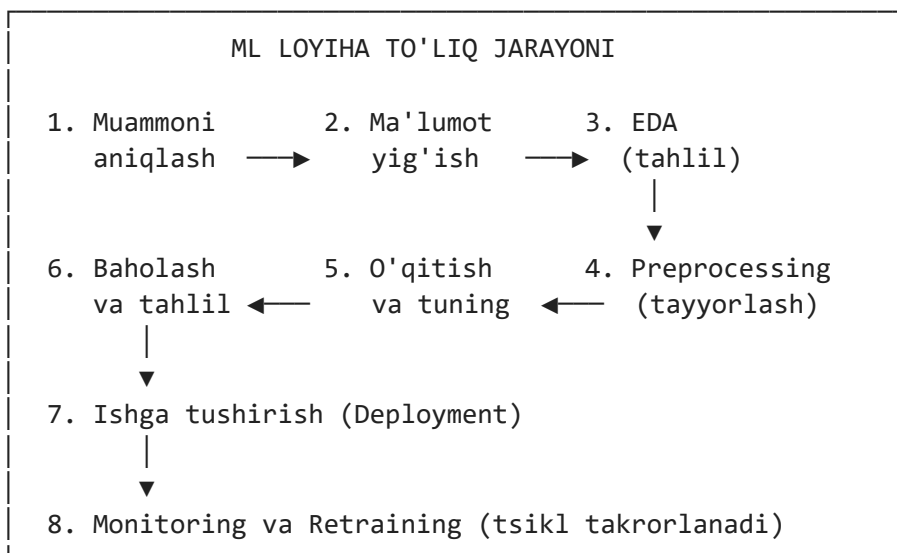
💡 **Eslab qol: Anscombe's Quartet** — statistik ko'rsatkichlari bir xil, lekin tuzilmasi butunlay boshqa 4 ta dataset bu haqiqatni isbotlaydi. `mean`, `std`, `corr` bir xil bo'lishi mumkin, lekin grafik chizilganda haqiqat ko'rinadi. **EDA dagi har bir vizualizatsiya — model uchun bitta muammoni oldini olish imkoni.** Ko'rmasdan qurilgan model — ko'r qurilgan uydek.

● SAVOL 45:

REAL LOYIHADA ML MODEL QURISH TO'LIQ JARAYONI: MUAMMONI ANIQLASH, MA'LUMOT YIG'ISH, EDA, PREPROCESSING, MODEL TANLASH, O'RGATISH, BAHOLASH VA ISHGA TUSHIRISH.

1. ML Loyiha Jarayonining Umumiy Ko'rinishi

Haqiqiy loyihalarda 80% vaqt ma'lumot bilan, faqat 20% vaqt model bilan o'tadi.



Bu jarayon – ITERATIV:
Har bosqichda muammo topilsa → oldingi bosqichga qaytiladi

2. 1-Bosqich: Muammoni Aniqlash

Bu bosqich texnik emas — **biznes va mantiq** bosqichi. Noto'g'ri savol → to'g'ri model ham foydasiz.

To'g'ri savol qo'yish:

NOTO'G'RI YONDASHUV:

"ML model qurañlik" → Keyin nima qilishni o'ylash
→ Maqsadsiz model → Hech kimga kerak emas

TO'G'RI YONDASHUV:

1. Biznes muammo nima?
"Mijozlar bankni tark etmoqda – ularni ushlab qolish kerak"
2. ML muammosi qanday?
"Mijoz keyingi 30 kunda ketadimi? → Klassifikatsiya"
3. Muvaffaqiyat qanday o'lchanadi?
"Ketuvchi mijozlarning 80% ni oldindan aniqlash ($\text{Recall} \geq 0.80$)"
4. Chiqish nima bo'lishi kerak?
"Har mijoz uchun ketish ehtimolligi 0-100%"
5. Qaror qanday qabul qilinadi?
"50% dan yuqori → retention campaign yuborish"

ML muammo turini aniqlash:

SAVOL	ML TURI
Qaysi sinf? (ha/yo'q)	→ Ikkilik klassifikatsiya
Qaysi sinf? (3 ta+)	→ Ko'p sinfli klassifikatsiya
Qancha? (son)	→ Regressiya
Qaysi guruhlar bor?	→ Klasterizatsiya (Unsupervised)
Keyingi qiymat nima?	→ Vaqt qatori (Time series)
Nima tavsiya qilinsin?	→ Tavsiya tizimi
Rasm ichida nima bor?	→ Computer Vision
Matn nimani anglatadi?	→ NLP

BIZNES MISOL → ML MUAMMO:

"Spam aniqlash" → Ikkilik klassifikatsiya
"Uy narxi" → Regressiya
"Mijoz segmentatsiya" → Klasterizatsiya
"Drone tahdid" → Object Detection (CV)
"Sotuv bashorat" → Time Series Regression

Muvaffaqiyat metrikalari — oldindan belgilash:

KLASSIFIKATSIYA:

Tibbiyot (kasallik): $\text{Recall} \geq 0.95$

→ FN (kasalni o'tkazib yuborish) qimmat!

Spam filtri: Precision ≥ 0.99
→ FP (muhim email spam bo'lishi) qimmat!

Teng muhim: F1-score yoki AUC-ROC

REGRESSIYA:

Narx bashorat: MAPE $\leq 10\%$ (foiz xato)
Fizik o'lchov: RMSE $\leq$ qabul qilingan chegara

MUHM QOIDA:

Metrikani OLDINDAN belgilash – model qurilgandan keyin emas! Aks holda "qulay" metrika tanlanib, natija sun'iy yuqori ko'rinadi.

3. 2-Bosqich: Ma'lumot Yig'ish

MA'LUMOT MANBALARI:

ICHKI manbalar:

Kompaniya DB (SQL)
CRM tizimi
ERP tizimi
Log fayllar
Sensor/IoT ma'lumoti

TASHQI manbalar:

Kaggle, UCI ML Repository
Government open data
API (Twitter, Weather)
Web scraping
Sotib olingan dataset

MA'LUMOT SIFATI TEKSHIRUVI:

Savol	Yaxshi	Muammo
Namunalar yetarlicha?	✓	< 100 → kam
Target (label) mavjudmi?	✓	Yo'q → UL
NaN foizi qancha?	< 5%	> 30% ⚠
Sinflar muvozanatlimi?	~50/50	99/1 ⚠
Ma'lumot yangi (fresh)?	✓	5 yil eski
Etik muammo yo'qmi?	✓	PII data ⚠

MINIMAL MA'LUMOT:

Oddiy klassifikatsiya: $\geq 1,000$ namuna
Murakkab klassifikatsiya: $\geq 10,000$ namuna
Deep Learning: $\geq 100,000$ namuna
Bu sonlar qat'iy emas – feature soni, murakkablikka bog'liq

4. 3-Bosqich: EDA (Exploratory Data Analysis)

Maqsad: Ma'lumotni modelga bergunga qadar **tushunish** — tuzilma, muammo, naqshlar.

EDA ketma-ketligi:

EDA BOSQICHLARI:

1. SHAKL VA TUR
df.shape, df.dtypes, df.info()
→ Qancha qator/ustun? Har ustun qanday tur?
2. NaN XARITASI
df.isnull().sum(), foiz hisoblash
→ Qaysi ustunlarda, qancha NaN?
3. TARGET TAHLILI
Klassifikatsiya: sinf nisbati (70/30? 99/1?)
Regressiya: target taqsimoti (skewed?)
4. FEATURE TAQSIMOTI
Har ustun: histogram, boxplot, skewness
→ Normal? Skewed? Bimodal?
5. OUTLIER ANIQLASH
Boxplot, z-score, scatter
→ Xato? Haqiqiy?
6. KORRELYATSIYA
df.corr(), heatmap
→ Target bilan bog'liq? Ustunlar o'rtasida?
7. KATEGORIK TAHLIL
value_counts(), barplot
→ Nechta noyob qiymat? Taqsimot?
8. MUNOSABAT TAHLILI
Pairplot, scatter matrix
→ Chiziqli? Nochiziqli? Ajralgan?

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
df = pd.read_csv('churn.csv') # Mijoz ketish dataset
```

```
# — 1. Dastlabki ko'rish —————
print(f"O'lcham: {df.shape}")
print(f"\nTurlar:\n{df.dtypes}")
print(f"\nNaN:\n{df.isnull().sum()}")
print(f"\nStatistika:\n{df.describe().round(2)}")
```

```
# — 2. Target taqsimoti —————
fig, axes = plt.subplots(2, 3, figsize=(15, 10))
```

```
churn_nisbat = df['Churn'].value_counts(normalize=True)
axes[0,0].pie(churn_nisbat.values,
              labels=['Qoldi', 'Ketdi'],
              autopct='%1.1f%%',
              colors=['steelblue', 'coral'])
axes[0,0].set_title('Target: Churn Nisbati')
# → 85/15 imbalanced!
```

```

# — 3. Son feature taqsimoti —————
son_ustunlar = df.select_dtypes(include=[np.number]).columns[:4]
for i, ustun in enumerate(son_ustunlar):
    ax = axes[0,1] if i==0 else axes[0,2] if i==1 \
        else axes[1,0] if i==2 else axes[1,1]
    sns.histplot(df[ustun], kde=True, ax=ax,
                 color='steelblue', alpha=0.7)
    skew = df[ustun].skew()
    ax.set_title(f'{ustun} (skew={skew:.2f})')
    if abs(skew) > 1:
        ax.set_facecolor('#fff3f3') # skewed → qizilish fon

# — 4. Korrelyatsiya heatmap —————
corr = df.select_dtypes(include=[np.number]).corr()
sns.heatmap(corr, annot=True, fmt='.2f',
            cmap='coolwarm', center=0,
            ax=axes[1,2], linewidths=0.5)
axes[1,2].set_title('Korrelyatsiya Heatmap')

plt.tight_layout()
plt.savefig('eda_report.png', dpi=150, bbox_inches='tight')
plt.show()

# — 5. Target bilan munosabat —————
fig, axes = plt.subplots(1, 3, figsize=(15, 4))

# Kategorik vs Target
sns.countplot(x='Contract', hue='Churn',
              data=df, ax=axes[0],
              palette={'No': 'steelblue', 'Yes': 'coral'})
axes[0].set_title('Shartnoma Turi va Churn')
# → Month-to-month → eng ko'p churn!

# Son vs Target
sns.boxplot(x='Churn', y='tenure',
            data=df, ax=axes[1],
            palette={'No': 'steelblue', 'Yes': 'coral'})
axes[1].set_title('Qolish Muddati va Churn')
# → Yangi mijozlar ko'proq ketadi!

# Korrelyatsiya bar
target_corr = corr['Churn_num'].drop('Churn_num') \
    .sort_values()
colors = ['coral' if v < 0 else 'steelblue'
          for v in target_corr]
axes[2].barh(target_corr.index, target_corr, color=colors)
axes[2].axvline(0, color='black', lw=0.8)
axes[2].set_title("'Churn' bilan Korrelyatsiya")

plt.tight_layout()
plt.show()

```

EDA xulosasi — qaror jadvali:

Target: 85/15	→ class_weight='balanced' ishlatish
tenure: right-skewed	→ log(tenure+1) transformation
TotalCharges: NaN 11	→ median bilan to'ldirish
Contract: 3 kategoriya	→ One-Hot Encoding
tenure ↔ TotalCharges	→ r=0.83 → multicollinearity!
	Birini o'chirish
MonthlyCharges	→ +0.19 → target bilan zaif bog'liq, saqlash

5. 4-Bosqich: Preprocessing

Maqsad: Ma'lumotni model qabul qila oladigan shaklga keltirish. EDA da topilgan muammolar shu yerda hal qilinadi.

PREPROCESSING PIPELINE:

Xom ma'lumot

- NaN to'ldirish
 - Son → GroupBy median
 - Kat → Mode / "Unknown"
- Yangi feature yaratish (Feature Engineering)
 - tenure_group = pd.cut(tenure, bins=[...])
 - charges_per_month = TotalCharges / tenure
- Kategorik kodlash
 - Ordinal → OrdinalEncoder
 - Nominal → OneHotEncoder (drop='first')
- Transformation
 - Skewed ustunlar → np.log1p()
- Feature selection
 - Korrelyatsiya < 0.05 → o'chirish
 - Multicollinearity > 0.85 → birini o'chirish
- Scaling
 - Linear/SVM/NN → StandardScaler
 - Neural Net → MinMaxScaler
 - Tree-based → Shart emas

```
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import (StandardScaler,
                                    OneHotEncoder,
                                    OrdinalEncoder)
from sklearn.model_selection import train_test_split
import pandas as pd
import numpy as np

df = pd.read_csv('churn.csv')
```

```

# — Feature Engineering —————
df['charges_per_month'] = np.where(
    df['tenure'] > 0,
    df['TotalCharges'] / df['tenure'],
    df['MonthlyCharges']
)
df['log_tenure'] = np.log1p(df['tenure'])

# — Target ajratish —————
X = df.drop(columns=['Churn', 'customerID'])
y = (df['Churn'] == 'Yes').astype(int)

# — AVVAL bo'lish, KEYIN scaling —————
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2,
    stratify=y,          # sinf nisbati saqlansin
    random_state=42
)

# — Ustunlarni ajratish —————
son_ustunlar = X.select_dtypes(
    include=[np.number]).columns.tolist()
nominal_kat  = ['InternetService', 'PaymentMethod']
ordinal_kat  = ['Contract']

# — Pipeline —————
son_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler())
])

nominal_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('encoder', OneHotEncoder(drop='first',
                               sparse_output=False))
])

ordinal_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('encoder', OrdinalEncoder(
        categories=[['Month-to-month',
                     'One year',
                     'Two year']]))
])

preprocessor = ColumnTransformer([
    ('son', son_pipeline, son_ustunlar),
    ('nominal', nominal_pipeline, nominal_kat),
    ('ordinal', ordinal_pipeline, ordinal_kat)
])

# Faqat fit – train dan!
X_train_proc = preprocessor.fit_transform(X_train)
X_test_proc  = preprocessor.transform(X_test)
print(f"Train: {X_train_proc.shape}")
print(f"Test: {X_test_proc.shape}")

```

6. 5-Bosqich: Model Tanlash va O'qitish

Model tanlash strategiyasi:

QOIDA: Sodda → Murakkab (Occam's Razor)

1. BASELINE MODEL
 - DummyClassifier (eng sodda, hech narsa o'rganmaydi)
 - Bu eng past chegara – har qanday model bundan yaxshi bo'lsin
2. ODDIY MODEL
 - Logistik Regressiya
 - Tez, tushunarli, kuchli baseline
3. TREE-BASED MODEL
 - Random Forest, XGBoost
 - Ko'p hollarda eng yaxshi natija
4. MURAKKAB MODEL
 - MLP, faqat zarur bo'lganda

TANLOV KRITERIYLARI:

Holat	Model
Interpretatsiya kerak (bank)	Logistik Regressiya (koeffitsientlar aniq ma'noli)
Tez natija kerak	Random Forest (default parametr)
Aniqlik muhim	XGBoost + Tuning
Ko'p ma'lumot, nochiziqli	MLP / Deep Learning

```
from sklearn.dummy import DummyClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (classification_report,
                             roc_auc_score,
                             f1_score)

import xgboost as xgb
import time

modellar = {
    'Baseline (Dummy)': DummyClassifier(
        strategy='most_frequent'),

    'Logistik Regressiya': LogisticRegression(
        class_weight='balanced',  # imbalanced uchun
        max_iter=1000,
        random_state=42),
```

```

'Random Forest': RandomForestClassifier(
    n_estimators=200,
    class_weight='balanced',
    random_state=42,
    n_jobs=-1),

'XGBoost': xgb.XGBClassifier(
    n_estimators=200,
    scale_pos_weight=5,          # imbalanced: 85/15 → ~5.6
    eval_metric='logloss',
    random_state=42,
    verbosity=0)
}

natijalar = {}
for nom, model in modellar.items():
    boshlanish = time.time()
    model.fit(X_train_proc, y_train)
    vaqt = time.time() - boshlanish

    y_pred = model.predict(X_test_proc)
    y_pred_proba = model.predict_proba(X_test_proc)[:,-1]

    natijalar[nom] = {
        'F1': f1_score(y_test, y_pred, average='weighted'),
        'AUC': roc_auc_score(y_test, y_pred_proba),
        'Vaqt': f"{vaqt:.2f}s"
    }

print(f"\n{'='*40}")
print(f"{nom}")
print(f"{'='*40}")
print(classification_report(y_test, y_pred,
    target_names=['Qoldi', 'Ketdi']))
print(f"AUC-ROC: {natijalar[nom]['AUC']:.4f}")
print(f"Vaqt: {natijalar[nom]['Vaqt']}")

```

```

# Natija qiyosi:
# Baseline:          AUC=0.50  F1=0.72  (kam)
# Logistik Regressiya: AUC=0.84  F1=0.79
# Random Forest:      AUC=0.91  F1=0.85
# XGBoost:            AUC=0.93  F1=0.87  ← eng yaxshi

```

Hyperparameter Tuning:

```

from sklearn.model_selection import RandomizedSearchCV
import xgboost as xgb

```

```

# XGBoost uchun parametrlar oraliq'i
param_tarqatish = {
    'n_estimators': [100, 200, 300, 500],
    'max_depth': [3, 4, 5, 6, 7],
    'learning_rate': [0.01, 0.05, 0.1, 0.2],
    'subsample': [0.6, 0.7, 0.8, 0.9, 1.0],
    'colsample_bytree': [0.6, 0.7, 0.8, 0.9, 1.0],
    'min_child_weight': [1, 3, 5]
}

```

```

}

xgb_model = xgb.XGBClassifier(
    scale_pos_weight=5,
    eval_metric='logloss',
    random_state=42,
    verbosity=0
)

# RandomizedSearch – GridSearch dan tezroq
qidiruv = RandomizedSearchCV(
    xgb_model,
    param_taqatish,
    n_iter=50,          # 50 ta tasodifiy kombinatsiya
    cv=5,              # 5-fold cross-validation
    scoring='roc_auc',
    n_jobs=-1,
    random_state=42,
    verbose=1
)

qidiruv.fit(X_train_proc, y_train)

print(f"Eng yaxshi parametrlar: {qidiruv.best_params_}")
print(f"Eng yaxshi AUC: {qidiruv.best_score_:.4f}")
eng_yaxshi = qidiruv.best_estimator_

```

7. 6-Bosqich: Baholash va Tahlil

```

import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import (confusion_matrix,
                             roc_curve,
                             precision_recall_curve)

import numpy as np

model = eng_yaxshi
y_pred = model.predict(X_test_proc)
y_pred_prob = model.predict_proba(X_test_proc)[:,1]

fig, axes = plt.subplots(2, 3, figsize=(18, 10))

# — 1. Confusion Matrix —————
cm = confusion_matrix(y_test, y_pred)
cm_foiz = cm / cm.sum(axis=1, keepdims=True) * 100
sns.heatmap(cm_foiz, annot=True, fmt='.1f',
            cmap='Blues', ax=axes[0,0],
            xticklabels=['Qoldi', 'Ketdi'],
            yticklabels=['Qoldi', 'Ketdi'])
axes[0,0].set_title('Confusion Matrix (%)')
axes[0,0].set_ylabel('Haqiqiy')
axes[0,0].set_xlabel('Bashorat')
# TN=92.3%, FP=7.7% | FN=14.2%, TP=85.8%
# → Recall(Ketdi)=85.8% → Maqsad ≥80% ✅

```

```

# — 2. ROC Curve —————
fpr, tpr, _ = roc_curve(y_test, y_pred_prob)
auc = roc_auc_score(y_test, y_pred_prob)
axes[0,1].plot(fpr, tpr, color='steelblue', lw=2,
               label=f'AUC = {auc:.3f}')
axes[0,1].plot([0,1],[0,1], '--', color='gray',
               label='Random (0.500)')
axes[0,1].fill_between(fpr, tpr, alpha=0.1, color='steelblue')
axes[0,1].set_title('ROC Curve')
axes[0,1].set_xlabel('False Positive Rate')
axes[0,1].set_ylabel('True Positive Rate')
axes[0,1].legend()

# — 3. Precision-Recall Curve —————
precision, recall, thresholds = precision_recall_curve(
    y_test, y_pred_prob)
axes[0,2].plot(recall, precision, color='coral', lw=2)
axes[0,2].axhline(y=0.5, linestyle='--',
                  color='gray', label='Baseline')
axes[0,2].set_title('Precision-Recall Curve')
axes[0,2].set_xlabel('Recall')
axes[0,2].set_ylabel('Precision')
# Imbalanced dataset da ROC dan muhimroq!

# — 4. Feature Importance —————
feature_nomlar = (son_ustunlar +
                  list(preprocessor.named_transformers_['nominal']
                      .named_steps['encoder']
                      .get_feature_names_out(nominal_kat)) +
                  ordinal_kat)
importances = model.feature_importances_
idx = np.argsort(importances)[-15:] # Top 15

axes[1,0].barh(range(len(idx)),
               importances[idx],
               color='steelblue', alpha=0.8)
axes[1,0].set_yticks(range(len(idx)))
axes[1,0].set_yticklabels([feature_nomlar[i] for i in idx])
axes[1,0].set_title('Top 15 Feature Importance')

# — 5. Threshold Tahlili —————
thresholds_arr = np.linspace(0.1, 0.9, 80)
f1_lar, precision_lar, recall_lar = [], [], []

for t in thresholds_arr:
    pred = (y_pred_prob >= t).astype(int)
    f1_lar.append(f1_score(y_test, pred,
                          zero_division=0))
    precision_lar.append(precision_score(y_test, pred,
                                         zero_division=0))
    recall_lar.append(recall_score(y_test, pred,
                                   zero_division=0))

eng_yaxshi_t = thresholds_arr[np.argmax(f1_lar)]
axes[1,1].plot(thresholds_arr, f1_lar,
               label='F1', color='steelblue', lw=2)
axes[1,1].plot(thresholds_arr, precision_lar,

```

```

        label='Precision', color='green', lw=2)
axes[1,1].plot(thresholds_arr, recall_lar,
               label='Recall', color='coral', lw=2)
axes[1,1].axvline(x=eng_yaxshi_t, color='black',
                  linestyle='--',
                  label=f'Best t={eng_yaxshi_t:.2f}')
axes[1,1].set_title('Threshold Tahlili')
axes[1,1].legend()
# → Default 0.5 emas, 0.38 da F1 maksimum!

# — 6. Learning Curve —————
from sklearn.model_selection import learning_curve
hajmlar, tr_ball, te_ball = learning_curve(
    eng_yaxshi, X_train_proc, y_train,
    train_sizes=np.linspace(0.1, 1.0, 8),
    cv=3, scoring='roc_auc', n_jobs=-1
)
axes[1,2].plot(hajmlar, tr_ball.mean(axis=1),
               'o-', color='steelblue', label='Train')
axes[1,2].plot(hajmlar, te_ball.mean(axis=1),
               'o-', color='coral', label='Validation')
axes[1,2].set_title('Learning Curve')
axes[1,2].set_xlabel('Train namunalar')
axes[1,2].set_ylabel('AUC-ROC')
axes[1,2].legend()

plt.suptitle('XGBoost Model Baholash', fontsize=14, y=1.01)
plt.tight_layout()
plt.savefig('model_evaluation.png', dpi=150,
            bbox_inches='tight')
plt.show()

```

8. 7-Bosqich: Ishga Tushirish (Deployment)

```

import joblib
import json
from datetime import datetime

# — Model saqlash —————
joblib.dump(preprocessor, 'preprocessor.pkl')
joblib.dump(eng_yaxshi, 'xgboost_churn.pkl')

# Model metadata
metadata = {
    'model_nomi': 'XGBoost Churn Classifier',
    'versiya': '1.0.0',
    'yaratilgan': datetime.now().isoformat(),
    'auc_roc': float(auc),
    'threshold': float(eng_yaxshi_t),
    'feature_lar': feature_nomlar,
    'train_namunalar': len(X_train),
    'test_namunalar': len(X_test)
}
with open('model_metadata.json', 'w') as f:
    json.dump(metadata, f, indent=2, ensure_ascii=False)

```

```

print("Model saqlandi: xgboost_churn.pkl")
# — Bashorat funksiyasi
import joblib
import pandas as pd
import numpy as np

class ChurnPredictor:
    """Production uchun wrapper klass"""

    def __init__(self, preprocessor_yol, model_yol,
                  metadata_yol):
        self.preprocessor = joblib.load(preprocessor_yol)
        self.model = joblib.load(model_yol)
        with open(metadata_yol) as f:
            self.meta = json.load(f)
        self.threshold = self.meta['threshold']

    def bashorat(self, mijoz_data: dict) -> dict:
        """
        Bitta mijoz uchun churn ehtimolligi.
        mijoz_data: {'tenure': 12, 'Contract': 'Month-to-month', ...}
        """
        df = pd.DataFrame([mijoz_data])
        X_proc = self.preprocessor.transform(df)
        ehtimol = self.model.predict_proba(X_proc)[0, 1]
        qaror = int(ehtimol >= self.threshold)

        return {
            'churn_ehtimoli': round(float(ehtimol), 4),
            'qaror': 'KETADI' if qaror else 'QOLADI',
            'xavf_darajasi': 'YUQORI' if ehtimol > 0.7
                           else 'O\'RTA' if ehtimol > 0.4
                           else 'PAST',
            'model_versiya': self.meta['versiya']
        }

# Ishlatish:
predictor = ChurnPredictor(
    'preprocessor.pkl',
    'xgboost_churn.pkl',
    'model_metadata.json'
)

yangi_mijoz = {
    'tenure': 3,
    'MonthlyCharges': 75.5,
    'TotalCharges': 226.5,
    'Contract': 'Month-to-month',
    'InternetService': 'Fiber optic',
    'PaymentMethod': 'Electronic check'
}

natija = predictor.bashorat(yangi_mijoz)
print(natija)
# {'churn_ehtimoli': 0.7821,
#   'qaror': 'KETADI',

```

```
# 'xavf_darajasi': 'YUQORI',  
# 'model_versiya': '1.0.0'}
```

9. 8-Bosqich: Monitoring va Retraining

DEPLOYMENT DAN KEYIN MUAMMOLAR:

DATA DRIFT:

Ma'lumot xususiyatlari o'zgaradi

Misol: Pandemiya → mijoz xulqi butun o'zgardi

→ Model 2019 da o'rgatilgan, 2021 da yomon ishlaydi

CONCEPT DRIFT:

Target munosabati o'zgaradi

Misol: "Spam" tushunchasi 5 yilda o'zgardi

→ Eski labellar yangi haqiqatni aks ettirmaydi

MODEL DEGRADATION:

Asta-sekin aniqlik pasayadi

→ Har oy AUC kuzatilsin

MONITORING TIZIMI:

Metrika	Ogohlantirish	Amal
AUC-ROC	< 0.85	Retraining
Precision	< 0.75	Threshold tuning
Recall	< 0.80	Retraining
Feature drift	PSI > 0.2	Feature check
Prediction drift	Taqsimot o'zgardi	Model yangilash

RETRAINING STRATEGIYALARI:

Davriy (Scheduled): Har oy yangi data bilan qayta o'qitish

Triggered: Aniqlik chegaradan tushsa → trigger

Online Learning: Har yangi ma'lumot → model yangilanadi
(Kafka, river kutubxonasi)

10. Xulosa

ML LOYIHA TO'LIQ JARAYONI:

1. MUAMMONI ANIQLASH

Biznes savol → ML savol

Metrikani OLDINDAN belgilash ($\text{Recall} \geq 0.80$)

ML turi: klassifikatsiya/regressiya/klaster?

2. MA'LUMOT YIG'ISH

Miqdor: ≥ 1000 (oddiy), ≥ 10000 (murakkab)

Sifat: NaN foizi, imbalance, yangilik

3. EDA

Taqsimot → transformation qaror
Outlier → tozalash qaror
Korrelyatsiya → feature tanlov qaror
Vizualizatsiya → MODEL qaror (chiziqli/yo'q)

4. PREPROCESSING (Pipeline!)

NaN → median/mode/guruh
Kategorik → OrdinalEncoder/OneHotEncoder
Scaling → StandardScaler/MinMaxScaler
AVVAL split, KEYIN fit_transform(train)!

5. MODEL TANLASH

Sodda → Murakkab (Occam's Razor)
Baseline → LogReg → RF → XGBoost → MLP

6. BAHOLASH

Confusion matrix, ROC, PR curve
Learning curve → overfit/underfit
Threshold tuning → biznes metrikasiga moslash

7. DEPLOYMENT

joblib.dump() → saqlash
Wrapper klass → ishlatish
Metadata → versiya, metrikalar

8. MONITORING

Data drift, concept drift kuzatish
Metrika pasaysa → retraining trigger

💡 **Eslab qol:** ML loyihada **80% vaqt — ma'lumot bilan** (yig'ish, EDA, preprocessing), faqat **20% — model bilan** o'tadi. Ko'pchilik buni teskari o'ylaydi. Eng kuchli model ham noto'g'ri tayyorlangan ma'lumot bilan yaxshi ishlamaydi — **"Garbage in, garbage out"** prinsipi ML da mutlaq haqiqat.